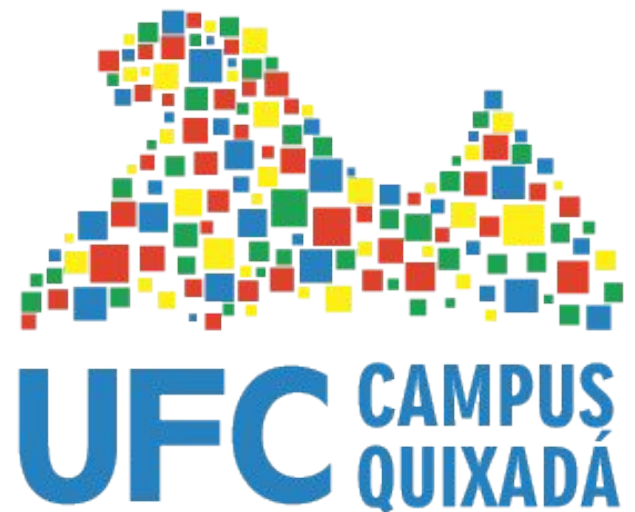


Arquitetura de Software

Exemplos de Arquiteturas

Sidarta Carvalho



Importância dos Padrões / Estilos

Slides baseados nos Capítulos 9 ao 18

O'REILLY®



Fundamentals of Software Architecture

An Engineering Approach

Mark Richards & Neal Ford

Importância dos Padrões / Estilos

- Sistemas que não possuem uma arquitetura bem definida são chamados de *Big Ball of Mud* / Código Spaguete / Megazorde

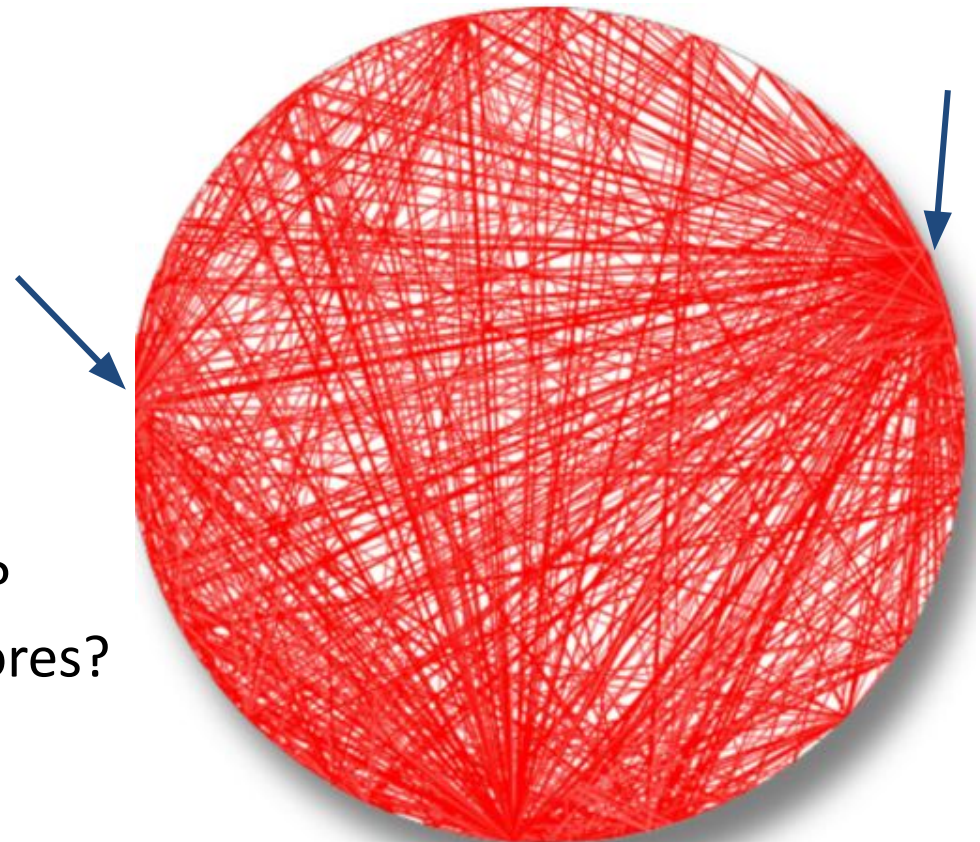


Importância dos Padrões / Estilos

- Este tipo de sistemas não estruturados possuem algumas características:
 - Crescimento desregulado / muito rápido
 - Reparos rápidos que são incorporados ao sistema
 - Desenvolvimento guiado à reparos
 - Ignorância das boas práticas de desenvolvimento
 - Acesso ao banco de dados direto da View
 - Não há ambiente de desenvolvimento, tudo é feito em produção
 - Testes só quando “der certo”
 - Alto acoplamento entre classes / funções / entidades

Importância dos Padrões / Estilos

- Exemplo de um sistema real baseado na web
 - Cada ponto é uma classe e as linhas são conexões entre as classes (usa ou é usada)



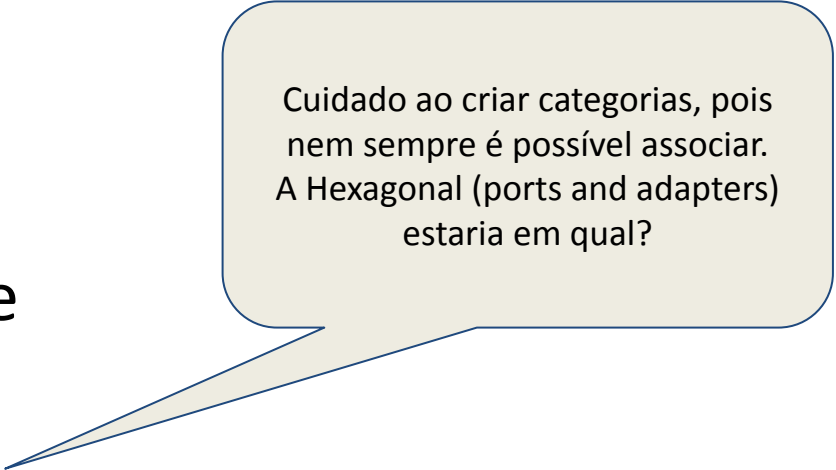
- É fácil corrigir bug?
- E incluir funcionalidades?
- Treinar novos colaboradores?

Importância dos Padrões / Estilos

- Exemplo de início de separação de software em camadas
 - é a criação da Arquitetura Cliente-Servidor, onde há uma distinção clara do *front-end* e do *back-end*
 - Depois a inclusão de camadas para persistência ou separação física do banco de dados
 - Após algum tempo mais camadas... como a camada de conexão na rede, onde era possível comunicar componentes usando troca de mensagens (CORBA / Chamada Remota de Procedimento - RPC)
 - Será que aqui começou-se a serializar Objetos para enviar na rede?

Classificação de Alguns Estilos / Padrões

- Monolithic
 - Layered architecture
 - Pipeline architecture
 - Microkernel architecture
- Distributed
 - Service-based architecture
 - Event-driven architecture
 - Service-oriented architecture
 - Microservices architecture



Cuidado ao criar categorias, pois nem sempre é possível associar. A Hexagonal (ports and adapters) estaria em qual?

Classificação de Alguns Estilos / Padrões

- Monolithic

- Layered architecture
- Pipeline architecture
- Microkernel architecture

Cuidado ao criar categorias, pois nem sempre é possível associar. A Hexagonal (ports and adapters) estaria em qual?

- Distributed

- Service-based architecture
- Event-driven architecture
- Service-oriented architecture
- Microservices architecture

Arquitetura Hexagonal é flexível e neutra em relação à topologia: Ela pode ser aplicada tanto em sistemas monolíticos quanto em distribuídos, e a decisão de onde aplicá-la depende das necessidades específicas de modularidade, isolamento e flexibilidade.

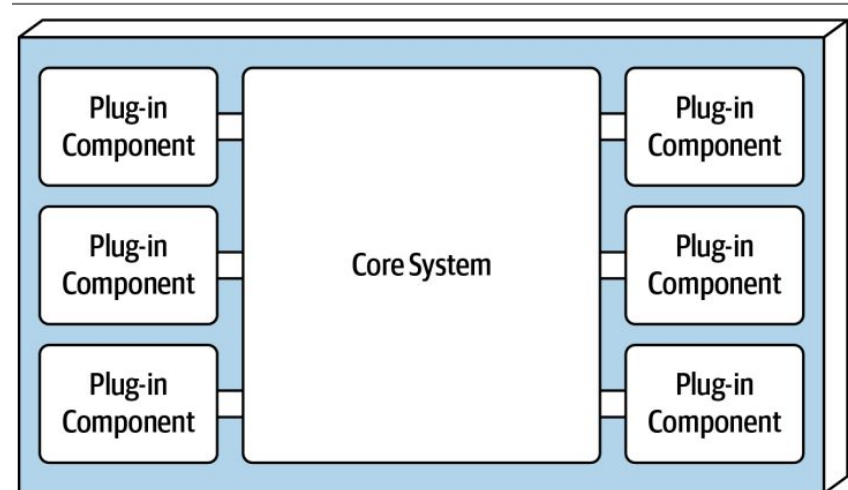
Classificação de Alguns Estilos / Padrões

- Um **sistema monolítico** é uma aplicação única e indivisível, onde todos os componentes estão integrados em um único bloco de código executável.
 - Ex.: interface, lógica de negócios, banco de dados
- Um **sistema distribuído** é composto por múltiplos componentes ou serviços que estão distribuídos em diferentes máquinas (físicas ou virtuais) e se comunicam por meio de redes.

Microkernel Architecture

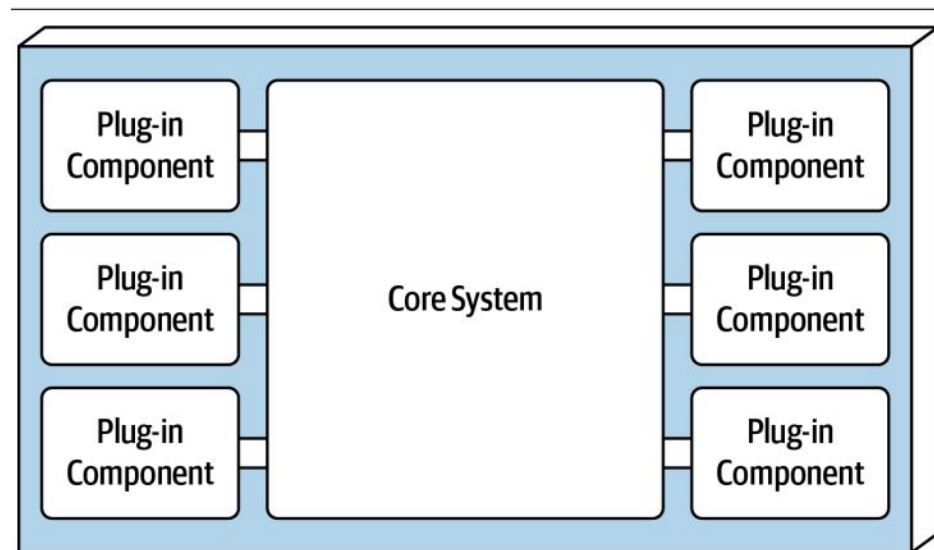
Microkernel Architecture

- Um dos padrões mais antigos, mas ainda utilizado atualmente.
- Uso geralmente para sistemas locais que são compilados e disponibilizados para download
- Composto por
 - Core system
 - Componentes plugáveis



Microkernel Architecture

- Os componentes permitem extensibilidade, adaptabilidade e isolamento do restante do sistema.
- O *core* do sistema possui parte da regra de negócio e o mínimo de código necessário para executar o sistema



Microkernel Architecture

- Eclipse IDE possui arquitetura de Microkernel, onde o Core é somente um editor de texto e todas as demais funcionalidades são plugins.
- Isso é tão verdade que tudo que é adicionado no Eclipse é plugin.
 - Antigamente, antes do AndroidStudio, desenvolver para Android era Eclipse + Plugin Android SDK.

Microkernel Architecture

- Por outro lado, se o Eclipse é uma IDE de programação, é aceitável que o Core fosse o conjunto de funcionalidades:
 - Editor texto, auto-complete código, compilação, etc.
- E os plugins fossem os componentes adicionais:
 - Suporte a uma nova linguagem ou plataforma de desenvolvimento, IA para autocompletar código, etc.

Microkernel Architecture

Exemplo prático

Microkernel Architecture

- Imagine que uma determinada aplicação precisa executar código distinto para cada plataforma de Hardware/Software.
- Exemplo de código Java

```
public void assessDevice(String deviceID) {  
    if (deviceID.equals("iPhone6s")) {  
        assessiPhone6s();  
    } else if (deviceID.equals("iPad1"))  
        assessiPad1();  
    } else if (deviceID.equals("Galaxy5"))  
        assessGalaxy5();  
    } else ...  
        ...  
    }  
}
```

Microkernel Architecture

- Ao invés de ter que colocar diversos if-else no código e precisar modificar o código-fonte a cada nova inclusão, é melhor criar componentes plugáveis para cada dispositivo.
- Além de melhorar a flexibilidade de adaptação e inclusão de novos componentes, separa-se as lógicas de negócio de cada dispositivo.

Microkernel Architecture

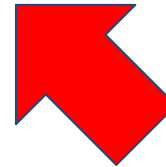
- Cada plugin contém suas próprias regras de negócio e está auto-contido, necessitando somente a atualização do código referente aos novos plugins.

```
public void assessDevice(String deviceID) {  
    String plugin = pluginRegistry.get(deviceID);  
    Class<?> theClass = Class.forName(plugin);  
    Constructor<?> constructor = theClass.getConstructor();  
    DevicePlugin devicePlugin =  
        (DevicePlugin)constructor.newInstance();  
    DevicePlugin.assess();  
}
```

Microkernel Architecture

- Cada plugin contém suas próprias regras de negócio e está auto-contido, necessitando somente a atualização do código referente aos novos plugins.

```
public void assessDevice(String deviceID) {  
    String plugin = pluginRegistry.get(deviceID);  
    Class<?> theClass = Class.forName(plugin);  
    Constructor<?> constructor = theClass.getConstructor();  
    DevicePlugin devicePlugin =  
        (DevicePlugin)constructor.newInstance();  
    DevicePlugin.assess();  
}
```



Padrão de projeto
Factory

- Explicando o Padrão Factory rapidamente...
 - Abstração da Criação de Objetos:
 - O método `assessDevice` não se preocupa diretamente com a instanciação de objetos específicos da classe `DevicePlugin`.
 - A criação do objeto é delegada para um método de fábrica (neste caso, implícito na lógica de obter a classe pelo nome e instanciá-la).
 - A complexidade da criação de objetos, que envolve reflexão para obter a classe pelo nome e instanciá-la, está encapsulada dentro do método.
 - Isso torna o código principal mais limpo e focado na sua responsabilidade principal.

Microkernel Architecture

- Programas como MacOS, Windows, VSCode e Eclipse IDE usam a arquitetura de Microkernel.
- O MacOS e Windows com um microkernel capaz de gerenciar arquivos, processos e coisas de baixo nível.
- O VSCode e Eclipse IDE apoiam a ideia de inclusão de novas linguagens, frameworks e funcionalidades através do uso de *plug-in*.

Microkernel Architecture

Benefício

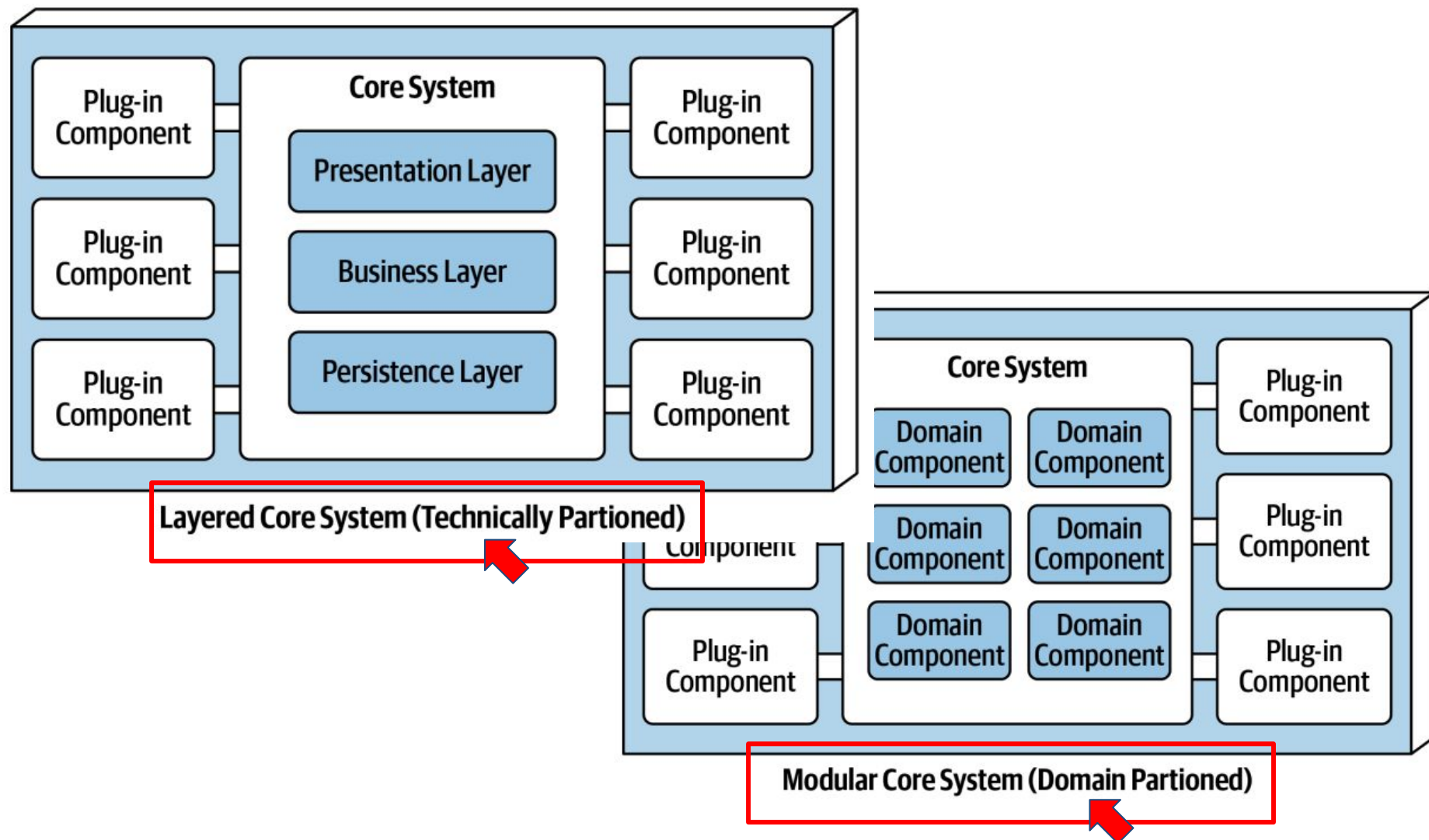
Extensibilidade	Qualquer um pode criar novas funcionalidades sem tocar no código-fonte do núcleo.
Isolamento	Uma extensão com erro não deve derrubar o editor inteiro.
Evolutividade	O núcleo pode ser atualizado sem quebrar as extensões, desde que mantenha as APIs estáveis.
Customização	Cada usuário monta seu próprio ambiente adicionando apenas os módulos (extensões) que deseja.

Microkernel Architecture

Parte 2

Microkernel Architecture

- Formas de implementar a arquitetura



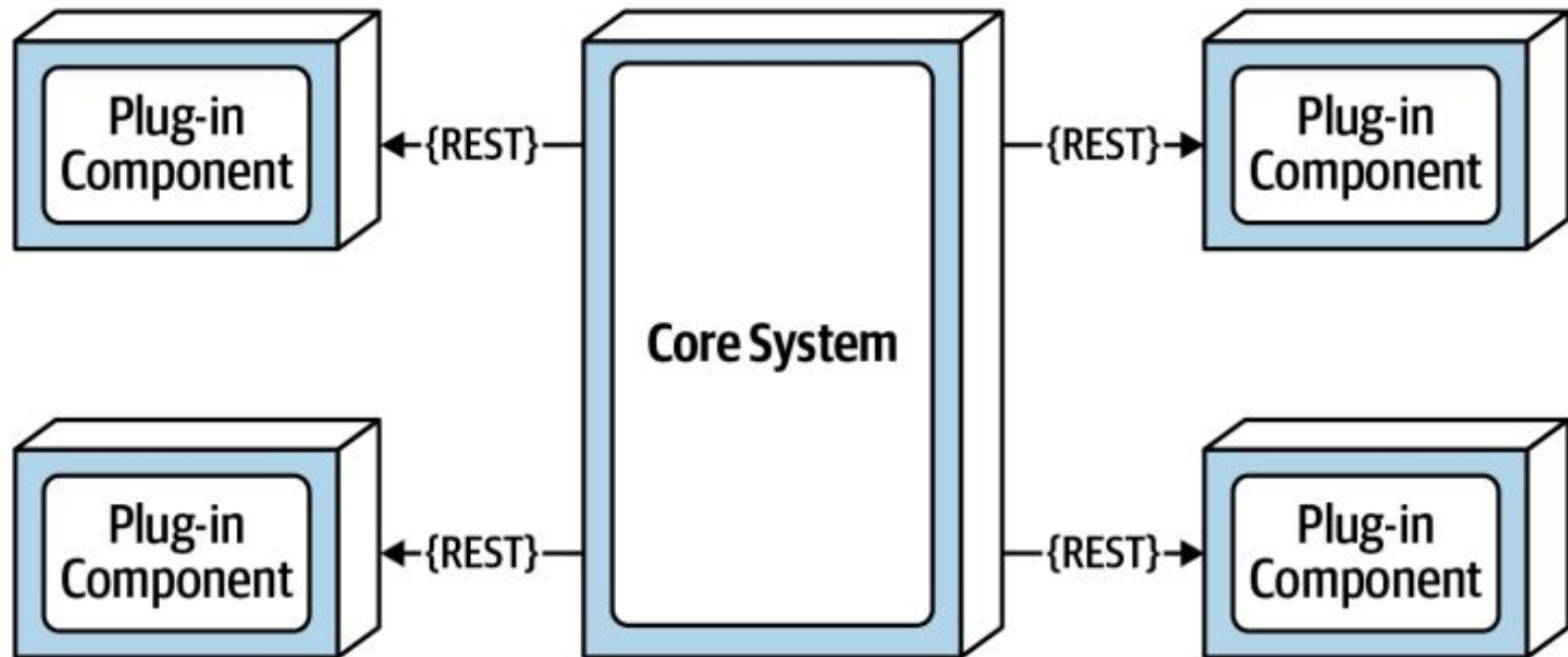
Microkernel Architecture

- Na separação técnica, há os diversos componentes que fornecem funcionalidades para todos os demais.
- Por exemplo, o componente de autenticação e logs é usado por toda a aplicação, independente do domínio do negócio.
- Na separação por negócio, cada “parte do negócio” gerencia seus próprios componentes.
- Por exemplo, componente Gerenciador PIX que possui autenticação, logs, comunicação com outros serviços, etc.

Microkernel Architecture

- Geralmente a comunicação do Core com os plugins é feita por chamada de métodos diretamente ou por instanciação dos plugins de forma dinâmica.
- Porém, também pode haver sistemas de Microkernel que utilizam requisições Web com HTTP e REST.
 - Isso melhora o desacoplamento dos componentes e permite a definição de interfaces ou padrões de comunicação entre os plugins.

Microkernel Architecture



Microkernel Architecture

- Por outro lado... hehehehe
- Fazer uso deste tipo de arquitetura com sistemas desacoplados e uma rede como meio de comunicação torna-o não mais Monolítico, mas Distribuído.
 - Mas mantendo os princípios do Microkernel

Microkernel Architecture

- Se os módulos REST estão apenas simulando a separação, mas fazem parte de um mesmo sistema lógico, você ainda tem um **Microkernel**, só que com **comunicação** distribuída.
- Se cada módulo REST é um serviço autônomo, com banco e ciclo de vida próprio, e o “núcleo” apenas coordena esses serviços, então o seu sistema evoluiu para uma **arquitetura distribuída** com princípios de microkernel (um híbrido).

Microkernel Architecture



Vamos esclarecer
alguns conceitos
nos próximos
slides!



Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★ ★ ★
Elasticity	★
Evolutionary	★ ★ ★
Fault tolerance	★
Modularity	★ ★ ★
Overall cost	★ ★ ★ ★ ★
Performance	★ ★ ★
Reliability	★ ★ ★
Scalability	★
Simplicity	★ ★ ★ ★
Testability	★ ★ ★

- *Architecture quantum*
 - Artefato implantável com grande coesão funcional entre os componentes e *synchronous connasence* (chamadas síncronas)
 - “Synchronous connasence implies synchronous calls within an application context or between distributed services that form this architecture quantum.
 - For example, if one service in a microservices architecture calls another one synchronously, each service cannot exhibit extreme differences in operational architecture characteristics.”

Termos relacionados

Aspecto	Escalabilidade	Elasticidade
Definição	Capacidade de lidar com o crescimento adicionando recursos.	Capacidade de ajustar os recursos dinamicamente.
Foco	Planejamento de capacidade a longo prazo.	Adaptação em tempo real às mudanças de carga de trabalho.
Natureza	Mudanças estáticas ou incrementais.	Ajustes dinâmicos e em tempo real.
Exemplo	Adicionar mais servidores para lidar com o crescimento.	Escalar instâncias para cima ou para baixo conforme a demanda.
Eficiência de custo	Pode ser caro se houver superdimensionamento.	Otimizado para cargas de trabalho flutuantes.

- Característica Evolutionary
 - Projetar e desenvolver novas soluções de forma iterativa, usando a arquitetura base.
 - Variedade de técnicas que podem ser combinadas para desenvolver uma solução final.
 - O conceito vem dos algoritmos / computação evolucionária.
 - Busca por soluções melhores a partir da mensuração de uma função de aptidão.

Microkernel Architecture

- *Maintainability* / Manutenibilidade inclui:
 - *Modularity* / Modularidade
 - Grau em que o software / sistema é composto de componentes discretos (individual e independente).
 - *Reusability* / Reusabilidade
 - Grau de reuso de componentes ou de criação de um novo a partir de um componente base.
 - *Analyzability*
 - Quão fácil é obter métricas sobre o software.

- *Maintainability* / Manutenibilidade inclui:
 - *Modifiability* / Modificabilidade
 - Grau em que pode-se alterar o software sem introduzir defeitos ou degradar o sistema atual.
 - Quão fácil é adicionar código sem quebrar / deteriorar o sistema.
 - *Testability* / Testabilidade
 - Quão fácil é testar o software.
 - Funções, componentes, integrações, etc.

Microkernel Architecture

- *Reliability / safety*
 - Quão seguro em termos de “fazer o que deve ser feito” é o sistema?
 - Se o sistema falhar, qual será o prejuízo?
 - Se envolver vidas, quanto isso poderia custar para a empresa?
- Performance / Desempenho
 - Testes de estresse, frequência de uso dos módulos e funções, tempos de resposta, etc

Microkernel Architecture

- Simplicidade e Custo Total são os principais pontos fortes da arquitetura Microkernel.
 - Custo Total remete a simplicidade de construir componentes e centralização de regras no *Core*.
- Escalabilidade, Tolerância a Falhas e Extensibilidade são as principais fraquezas.
 - Que estão relacionadas a arquitetura Monolítica...

Microkernel Architecture

- Além disso, similar a Arquitetura em Camadas, o número *quanta* é sempre 1, pois todas as requisições devem passar pelo *Core* porque os plugins são independentes.
 - Sendo o *Core* responsável por gerenciar a comunicação e a ordem das execuções.

Microkernel Architecture

- Outra característica marcante da Arquitetura Microkernel é poder ser particionada por Domínio ou de forma Técnica sendo a maioria particionada por Domínio.
- Exemplo:
 - Sistemas que dependem de localização ou tipo de cliente possuem uma boa aderência com a arquitetura Microkernel
 - É possível criar plugins para diferentes regiões e associá-las ao *Core*, contendo as regras de negócio que são independentes de localização.
- Sobre Desempenho / Performance pode-se pensar que a aplicação pode ter plugins removidos para deixá-la mais rápida.

Microkernel Architecture

- Por fim, a Disponibilidade / Availability pode ser afetada pois o mean-time-to-recovery (MTTR) é geralmente alto, por conta de todo o *Core* e plugins que precisam ser carregados e sincronizados.
- Usualmente, o início (*start* do executável) de aplicações com Arquitetura Microkernel variam de 2 a 20 minutos para grandes aplicações.

Dúvidas?



www.shutterstock.com · 744867163

Hexagonal Architecture (Ports and Adapters)

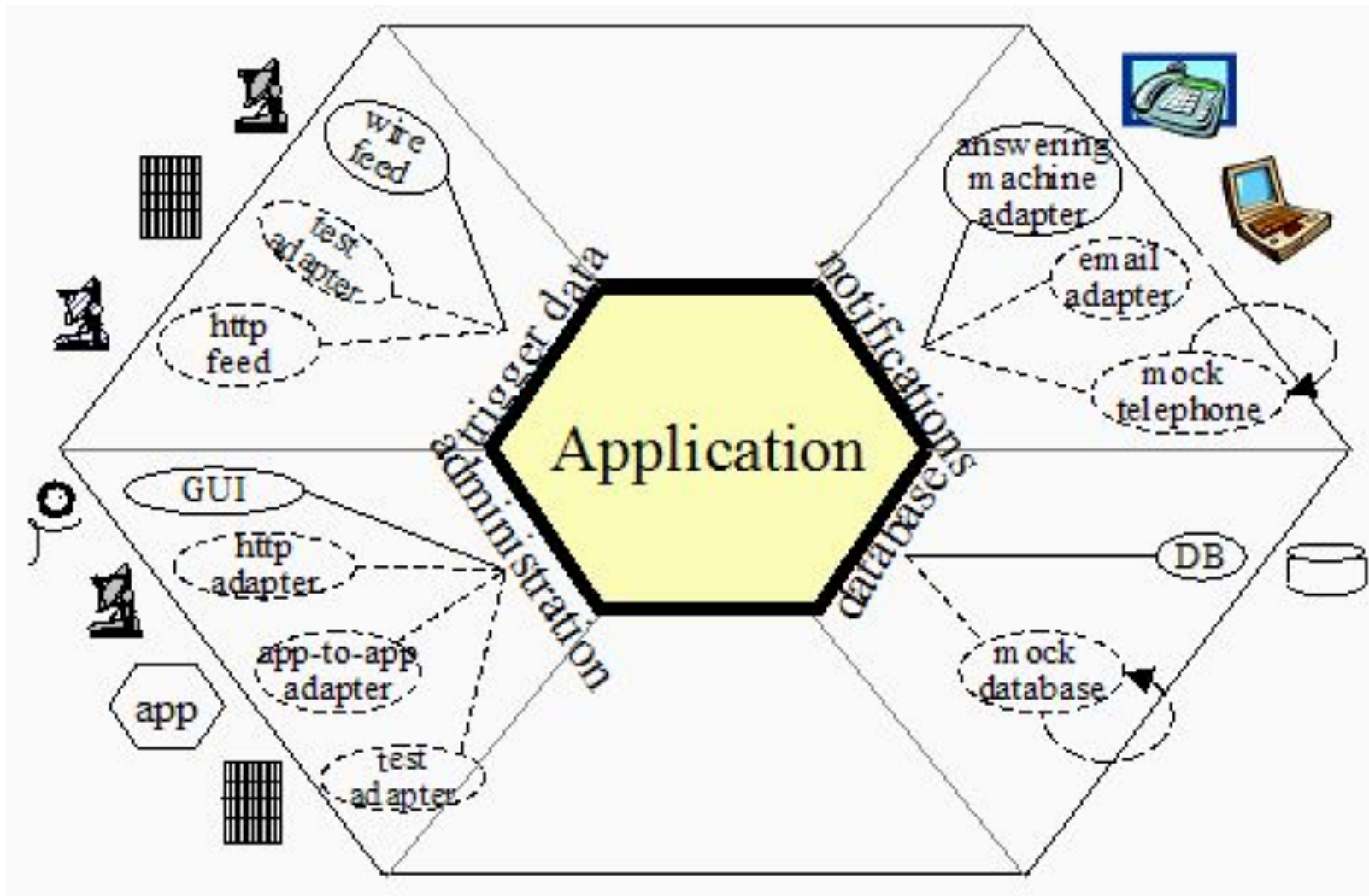
Não tem no livro

Hexagonal Architecture (Ports and Adapters)

- Organiza o sistema em torno de um núcleo independente que interage com o mundo externo por meio de interfaces (portas) e implementações específicas (adaptadores)
 - Facilitando substituições e integrações sem impactar a lógica central
- Criada em 2005
 - Recente quando comparada a arquitetura em camadas (1960~1970)
 - Arquitetura em camadas popularizou-se com o Modelo OSI - TCP/IP

<https://alistair.cockburn.us/hexagonal-architecture/>

Hexagonal Architecture (Ports and Adapters)



<https://alistair.cockburn.us/hexagonal-architecture/>

Hexagonal Architecture (Ports and Adapters)

- Permite que atores externos comuniquem com o *core* da aplicação usando adaptadores (interfaces) independente do que implementa a interface.
- Por exemplo, uma porta USB permite conexão de múltiplos dispositivos pela mesma interface.
- As portas auxiliam a encapsular o domínio da aplicação.
- As portas da arquitetura também permitem a comunicação da aplicação com:
 - sistemas ou serviços externos
 - bancos de dados
 - APIs, etc.

Rápida comparação...

Hexagonal Architecture vs MicroKernel

- A princípio, similar a Microkernel, mas há algumas diferenças marcantes.
- Microkernel
 - Tornar o sistema modular e extensível, permitindo adicionar novas funcionalidades (plugins) sem alterar o núcleo.
- Hexagonal (Ports & Adapters)
 - Tornar o sistema independente de frameworks, bancos e interfaces externas, isolando o domínio da infraestrutura.

No **Microkernel**, os plugins **adicionam novas capacidades** ao sistema (novas funções). Na **Hexagonal**, os adaptadores **mudam o meio de comunicação** com o mesmo domínio (novos meios de entrada/saída).

Hexagonal Architecture (Ports and Adapters)

Sistema de pagamentos

Microkernel

Core

Processa pedidos genéricos.

Plugins

Adicionam novos tipos de pagamento (Pix, Cartão, Boleto).

O sistema ganha novas funcionalidades de negócio.

Hexagonal

Domínio

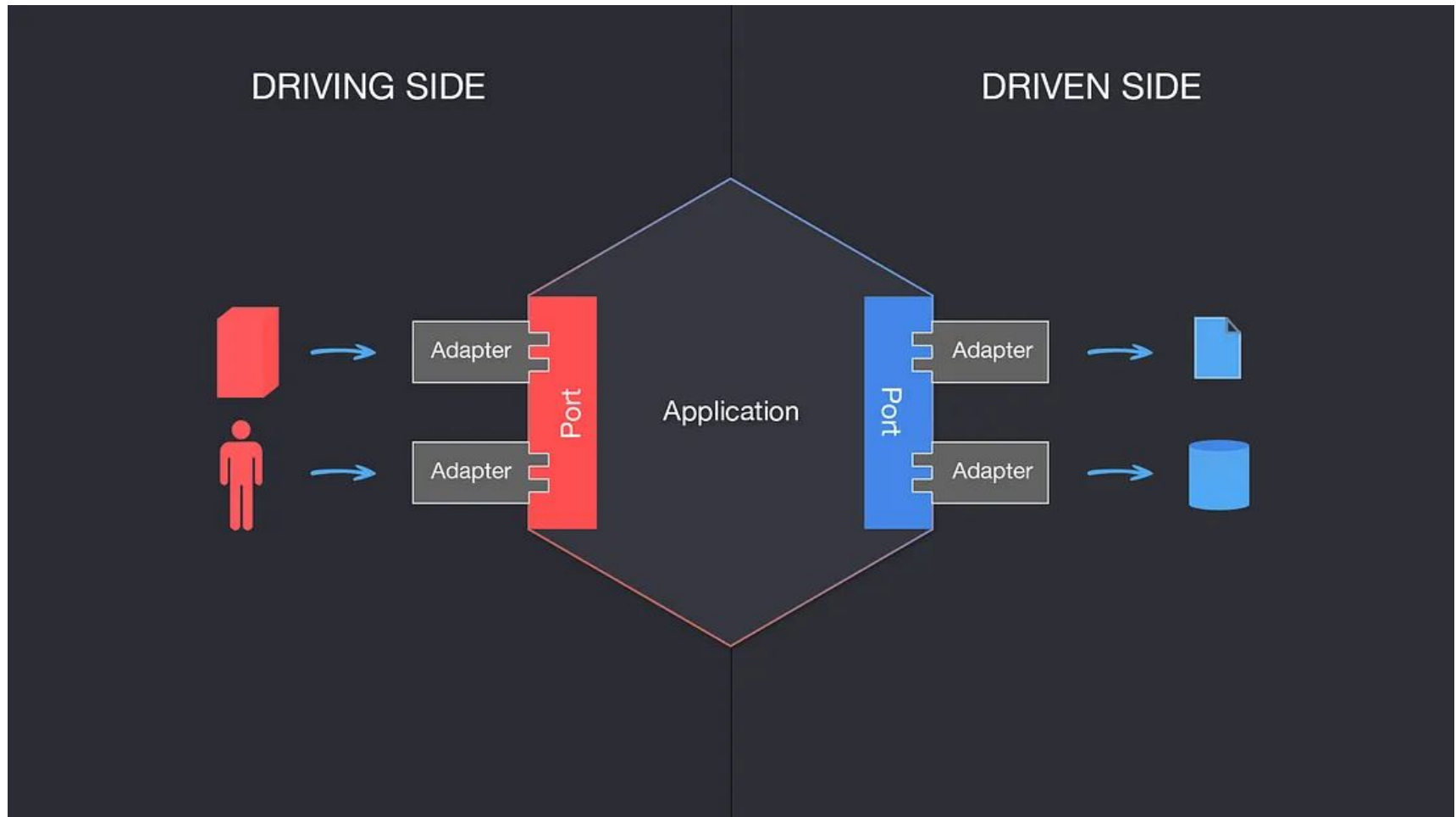
Regras de pagamento (autenticação, validação, cobrança).

Adaptadores

REST (entrada), Banco de dados (saída), Mensageria (saída).

O sistema mantém o mesmo domínio, mas pode trocar tecnologias (REST ↔ gRPC, MySQL ↔ MongoDB).

Hexagonal Architecture (Ports and Adapters)



<https://medium.com/ssense-tech/hexagonal-architecture-there-are-always-two-sides-to-every-story-bc0780ed7d9c>

Hexagonal Architecture: Vantagens

- Independência de Tecnologia
 - A lógica de negócios é isolada de frameworks, bancos de dados e interfaces de usuário, permitindo mudanças nas tecnologias externas sem impacto no núcleo do sistema.
 - Proteção ao negócio
- Facilidade de Testes
 - Com interfaces bem definidas para entrada e saída, é mais fácil realizar testes unitários e de integração, pois o núcleo pode ser testado de forma independente dos adaptadores externos.
- Alta Flexibilidade e Extensibilidade
 - Novos adaptadores (como diferentes APIs ou bancos de dados) podem ser adicionados ou substituídos sem modificar o núcleo, facilitando a evolução do sistema.

Hexagonal Architecture: Vantagens

- Manutenibilidade
 - A separação clara entre lógica de negócios e infraestruturas externas reduz o acoplamento, tornando o código mais modular e fácil de manter.
- Escalabilidade
 - A arquitetura facilita a transformação de um sistema monolítico em uma arquitetura distribuída ou baseada em microsserviços, caso seja necessário escalar o sistema.

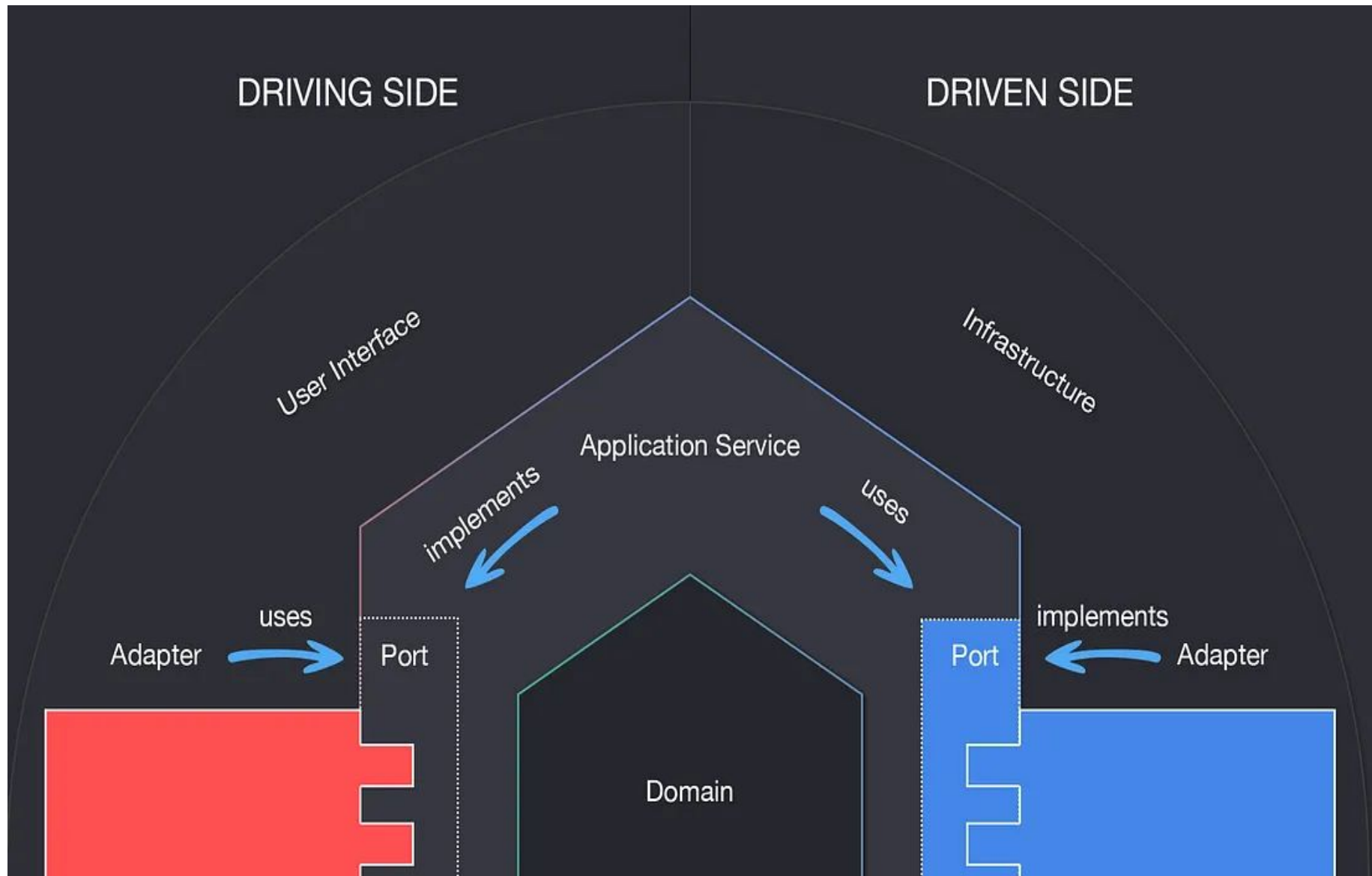
Hexagonal Architecture: Coisas não tão boas...

- Complexidade Inicial
 - Pode ser complexo no início, especialmente para projetos pequenos, já que envolve o uso de interfaces e múltiplos componentes para abstrair dependências.
- Sobrecarga de Código
 - A separação em portas e adaptadores (interfaces) para cada funcionalidade externa pode resultar em mais classes e código boilerplate, o que pode tornar o projeto mais *verboso* e reduzir a manutenibilidade.

Hexagonal Architecture: Coisas não tão boas...

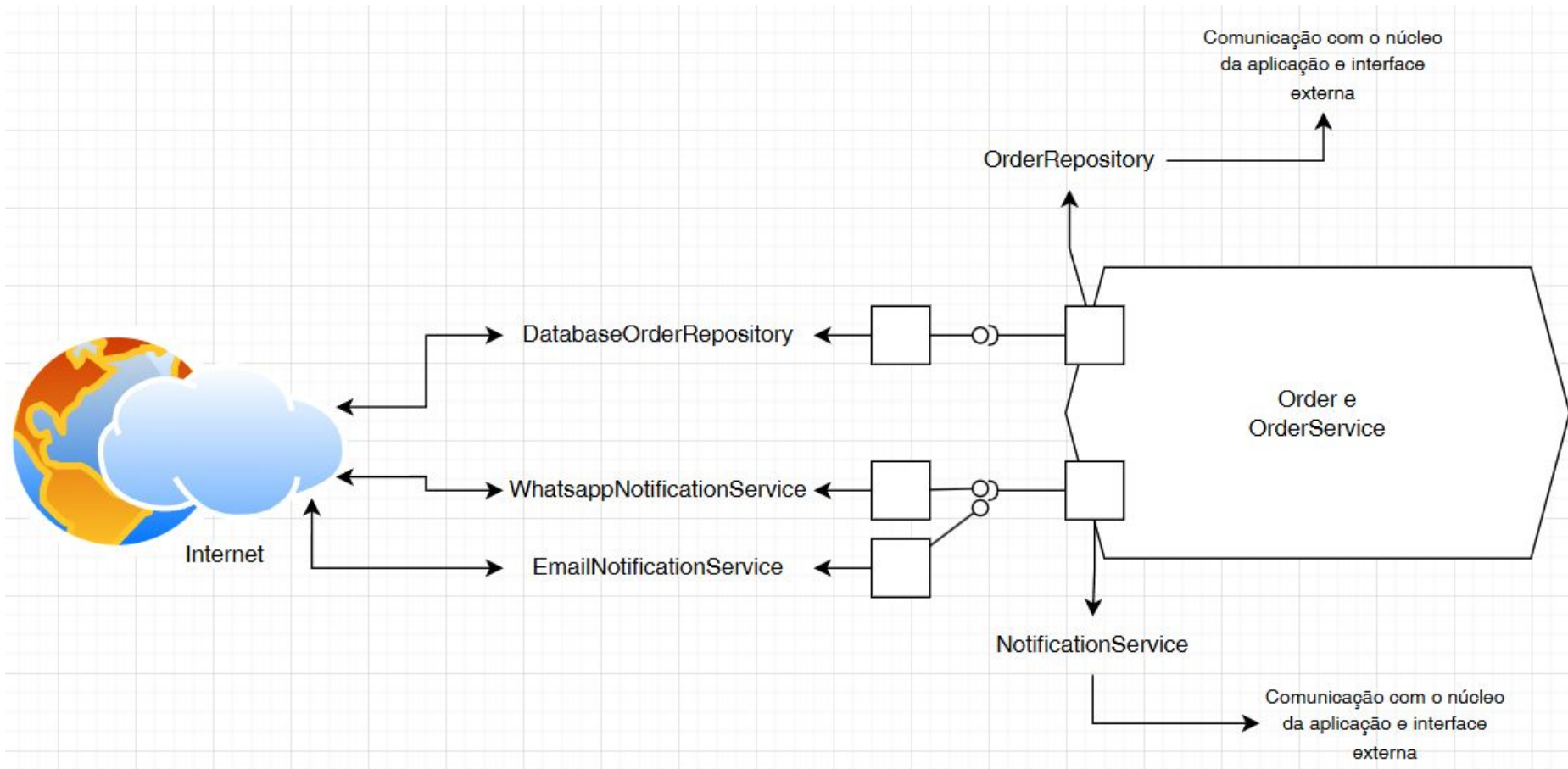
- Maior Custo de Manutenção
 - Como o padrão impõe um nível de abstração elevado, mudanças significativas nos requisitos de negócios podem exigir modificações tanto no núcleo como nos adaptadores, elevando o custo de manutenção em algumas situações.
 - Devo usar em um projeto que pode mudar as regras de negócio com frequência?
- *Overengineering*
 - Em projetos que não demandam alta flexibilidade ou modularidade, aplicar a Arquitetura Hexagonal pode levar a um excesso de abstrações, o que pode ser um esforço desproporcional e não trazer benefícios claros para o projeto.

Hexagonal Architecture (Ports and Adapters)



Hexagonal Architecture (Ports and Adapters)

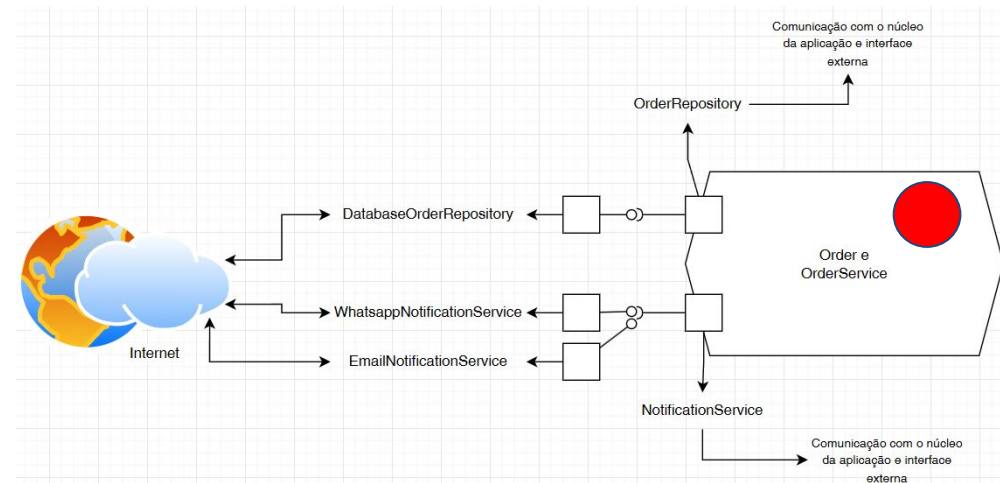
<https://app.diagrams.net/#G1AmZljEWYWa0cSC4MIM0NmJOiwEYMyDZx#%7B%22pageId%22%3A%22PSgiV7mgtBJI5Bp8Vi2f%22%7D>



Hexagonal Architecture (Ports and Adapters)

```
public class Order {  
    private String id;  
    private String product;  
    private int quantity;  
    private String customerEmail;  
  
    public Order(String id, String product, int  
quantity, String customerEmail) {  
        this.id = id;  
        this.product = product;  
        this.quantity = quantity;  
        this.customerEmail = customerEmail;  
    }  
  
    // Getters e Setters  
}
```

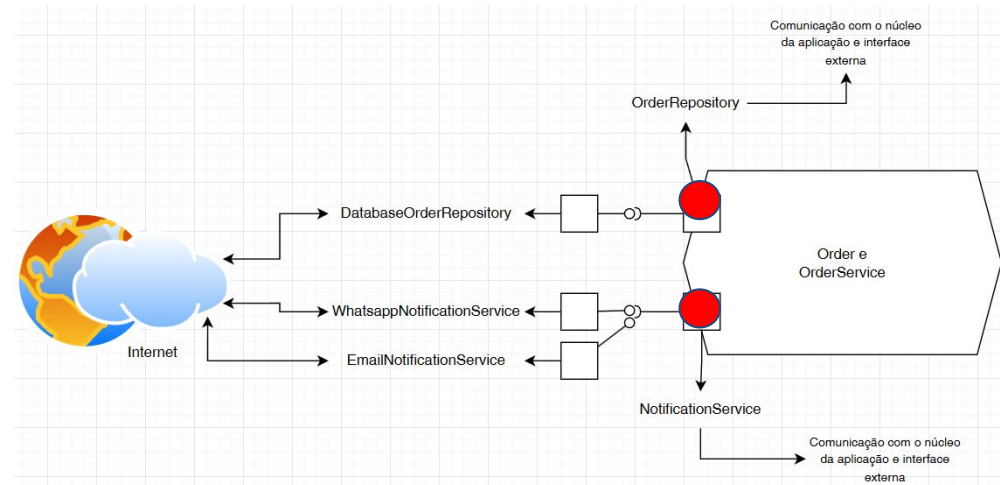
```
public class OrderService {  
    private final OrderRepository orderRepository;  
    private final NotificationService notificationService;  
  
    public OrderService(OrderRepository orderRepository,  
NotificationService notificationService) {  
        this.orderRepository = orderRepository;  
        this.notificationService = notificationService;  
    }  
  
    public void placeOrder(Order order) {  
        orderRepository.save(order);  
        notificationService.sendOrderConfirmation(order);  
    }  
}
```



Hexagonal Architecture (Ports and Adapters)

```
public interface OrderRepository {  
    void save(Order order);  
    Order findById(String orderId);  
}
```

```
public interface NotificationService {  
    void sendOrderConfirmation(Order order);  
}
```



```

public class DatabaseOrderRepository implements OrderRepository {
    // Aqui, poderia usar uma conexão com banco de dados ou ORM, como JPA/Hibernate

    @Override
    public void save(Order order) {
        System.out.println("Salvando pedido no banco de dados: " + order.getId());
        // Implementação de persistência no banco
    }

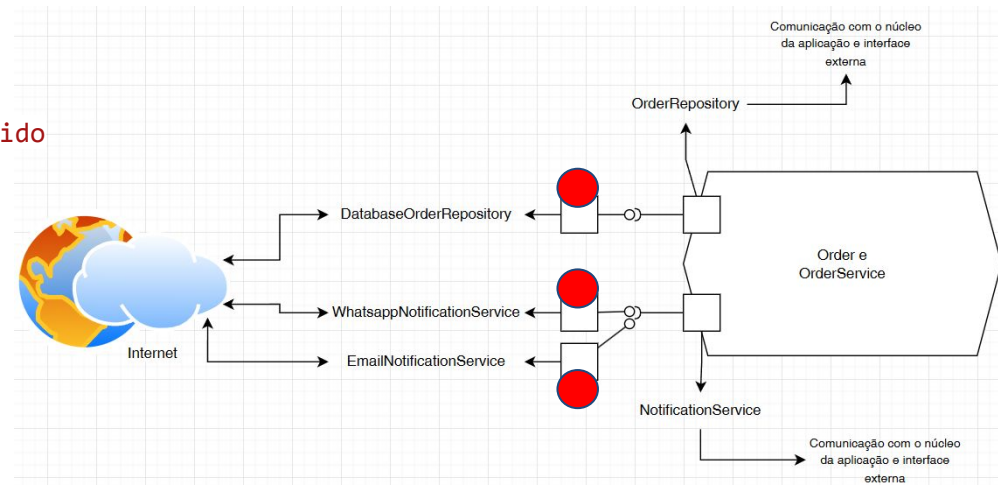
    @Override
    public Order findById(String orderId) {
        System.out.println("Buscando pedido no banco de dados: " + orderId);
        // Implementação de busca no banco
        return null;
    }
}

```

```

public class EmailNotificationService implements
NotificationService {
    @Override
    public void sendOrderConfirmation(Order order) {
        System.out.println("Enviando confirmação de pedido
para: " + order.getCustomerEmail());
        // Implementação de envio de e-mail
    }
}

```



Hexagonal Architecture (Ports and Adapters)

```
public class Main {  
    public static void main(String[] args) {  
        // Configuração de dependências  
        OrderRepository orderRepository = new DatabaseOrderRepository();  
        NotificationService notificationService = new EmailNotificationService();  
  
        // Inicialização do serviço de pedidos  
        OrderService orderService = new OrderService(orderRepository, notificationService);  
  
        // Criando um pedido de exemplo  
        Order order = new Order("1", "Laptop", 1, "cliente@exemplo.com");  
  
        // Processando o pedido  
        orderService.placeOrder(order);  
    }  
}
```

Atividade

- Você é o tech lead do time de desenvolvedores do *Chicken Bank of America*. Ao iniciar as operações do banco no Brasil, foi necessário a inclusão do sistema PIX no seu portfólio de produtos.

Crie uma arquitetura simplificada do sistema PIX usando a arquitetura Hexagonal no Draw.io.



Service-Based Architecture

Service-Based Architecture

- Service-Based vs Microserviços
 - Similar, mas diferente!
- Ambas envolvem a criação de sistemas compostos por serviços, mas divergem principalmente em
 - Granularidade
 - Acoplamento
 - Escalabilidade
 - Comunicação

- Microserviços:
 - Cada serviço é extremamente pequeno e realiza uma única função ou responsabilidade (princípio do Single Responsibility).
 - São projetados para serem altamente especializados e autônomos.
 - Exemplos
 - Um serviço apenas para autenticação por email e senha, outro para login social, outro para pagamento por cartão de crédito, etc.
- Service-Based
 - Os serviços são maiores e abrangem múltiplas responsabilidades relacionadas a um domínio específico.
 - Um único serviço pode conter diversas funcionalidades relacionadas.
 - Exemplos: um serviço completo de "Gestão de Clientes" que inclui autenticação, perfil e histórico de compras.

- Microserviços:
 - Princípio da Responsabilidade Única (SRP)
 - Totalmente independentes em termos de desenvolvimento, implantação e manutenção.
 - Podem ser desenvolvidos e implantados em diferentes linguagens de programação ou plataformas.
 - Comunicação entre microserviços geralmente ocorre via APIs REST ou mensagens assíncronas.
- Service-Based:
 - Os serviços têm menor nível de independência.
 - Geralmente, são parte de uma implantação monolítica maior, ainda que sejam logicamente separados.
 - A comunicação entre serviços pode ser mais acoplada e frequentemente dentro do mesmo ambiente de execução.
 - Geralmente usada como transição entre monolítico modular e microserviços.

- Microserviços:
 - Escalabilidade granular
 - apenas os serviços que exigem mais recursos são escalados.
 - Ideal para lidar com diferentes taxas de carga em partes específicas do sistema.
- Service-Based:
 - Escalabilidade mais limitada, já que os serviços costumam ser mais monolíticos e menos independentes.
 - Escalar um serviço pode exigir escalar toda a aplicação ou partes relacionadas.

- Microserviços:
 - Comunicação é mais complexa, baseada em APIs REST ou mensagens assíncronas
 - Ex.: RabbitMQ e Kafka
 - Projetados para funcionarem mesmo em ambientes distribuídos (diferentes servidores ou nuvens).
- Service-Based
 - Comunicação tende a ser mais direta e dentro de uma infraestrutura comum.
 - Geralmente é síncrona
 - chamadas de procedimento remoto ou invocações internas.

Service-Based Architecture

Aspecto	Microserviços	Service-Based
Granularidade	♦ Fina — serviços pequenos e especializados	♦ Grossa — serviços maiores, com várias responsabilidades
Independência	🧩 Totalmente independentes (deploy isolado)	🔗 Interdependentes, geralmente implantados juntos
Escalabilidade	📈 Escalável de forma granular (por serviço)	📊 Escalável, mas depende de serviços associados
Comunicação	🌐 REST APIs / mensagens assíncronas (Kafka, RabbitMQ)	🔄 Comunicação local e síncrona (chamadas internas)
Complexidade Operacional	⚡ Alta — requer orquestração e observabilidade	🧱 Moderada — mais simples de gerenciar
Exemplos de Uso	🚀 Netflix, Uber, Amazon	🏢 Sistemas corporativos internos, ERPs médios

Service-Based vs Microserviços

Imagine um sistema de e-commerce tradicional dividido em serviços grandes por domínio.

Serviço	Responsabilidades
Serviço de Clientes	Cadastro, autenticação, atualização de perfil, histórico de compras
Serviço de Produtos	Cadastro de produtos, controle de estoque, categorias
Serviço de Vendas	Carrinho, processamento de pedidos, emissão de notas
Serviço de Pagamentos	Processamento de cartão, boleto, integração com gateways

Microserviço	Responsabilidade única
Auth-Service	Login, logout, tokens JWT
User-Profile-Service	Dados do usuário, endereço, preferências
Product-Catalog-Service	Lista de produtos e categorias
Inventory-Service	Controle de estoque
Order-Service	Criação e rastreamento de pedidos
Payment-Service	Processamento de pagamentos
Notification-Service	Envio de emails e SMS

Service-Based Architecture

- A Arquitetura Baseada em Serviços pode ser “quebrada” de várias formas, pode-se separar por
 - Domínio ou funcionalidades
 - Interfaces do usuário
 - como o usuário interage com o serviços
 - Bancos de dados
 - etc

Service-Based Architecture

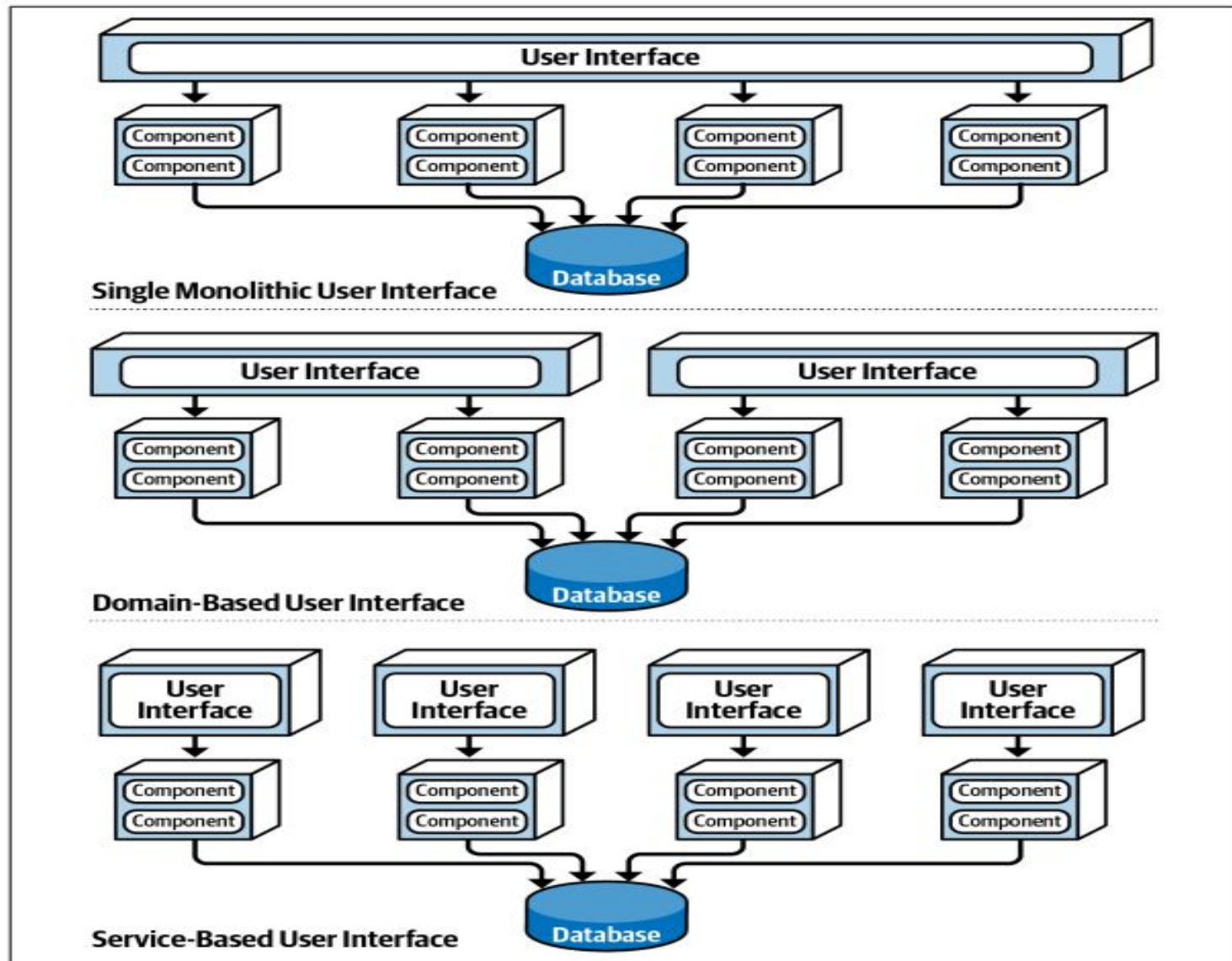


Figure 13-2. User interface variants

Service-Based Architecture

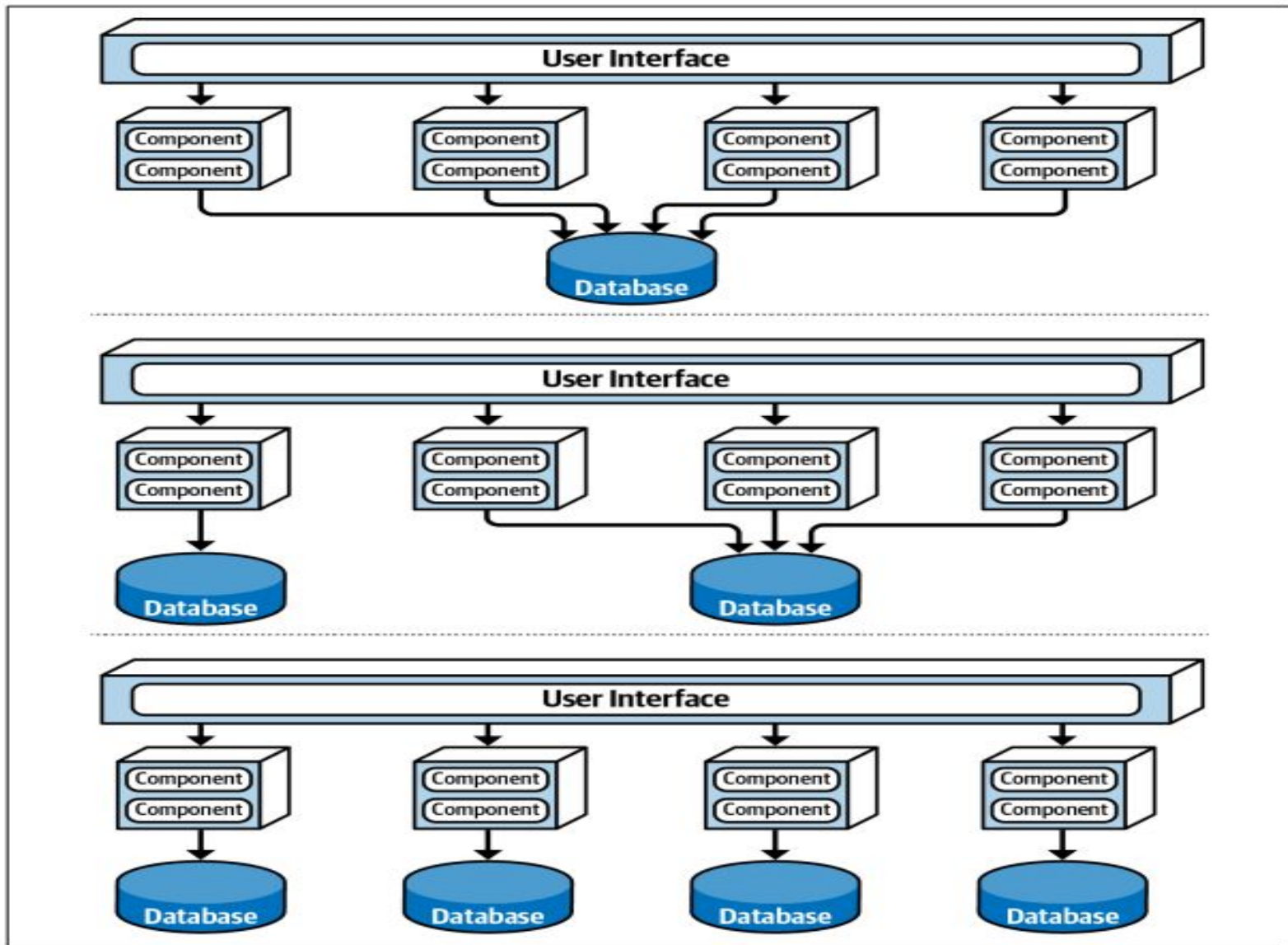


Figure 13-3. Database variants

Service-Based Architecture

- Para não permitir que o usuário acesse diretamente a camada de negócio dos serviços, ou seja, acesse diretamente o serviço, pode-se incluir uma camada de *Gateway*

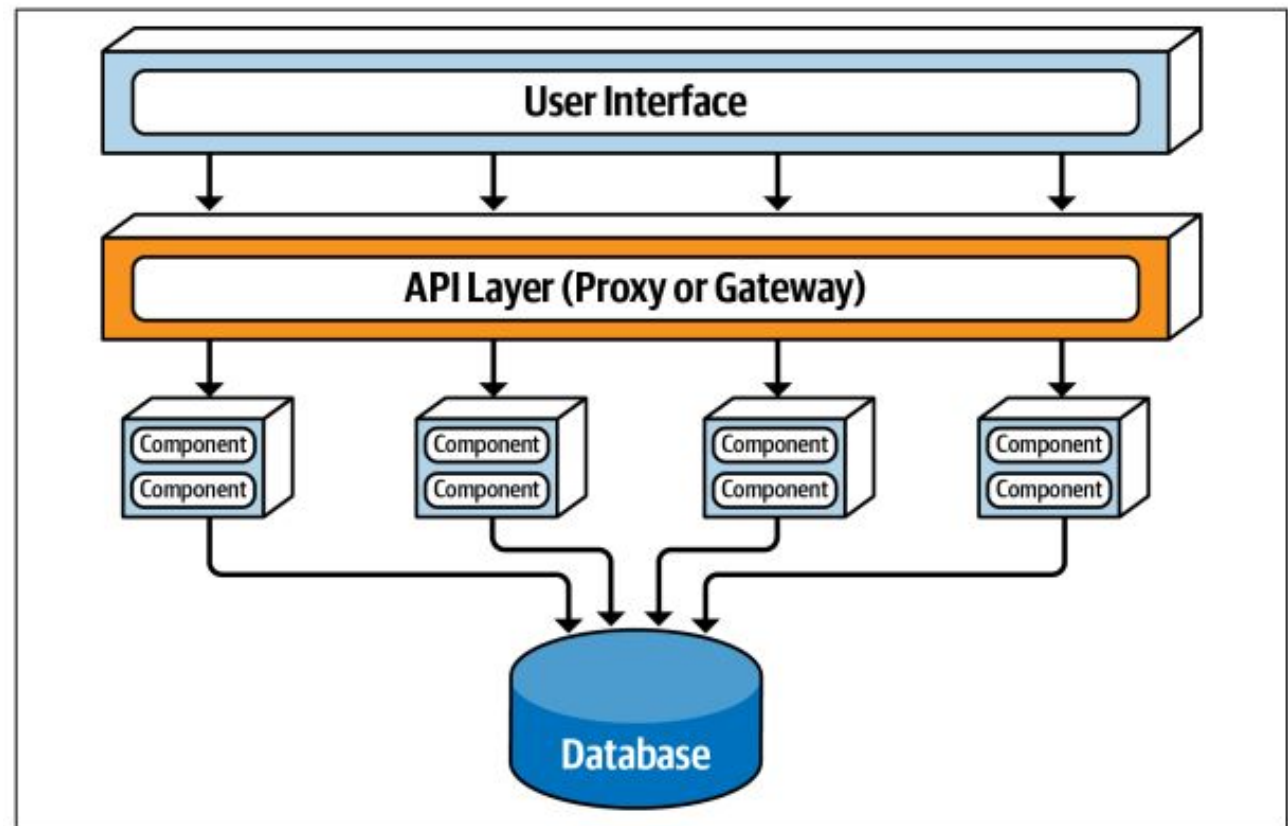
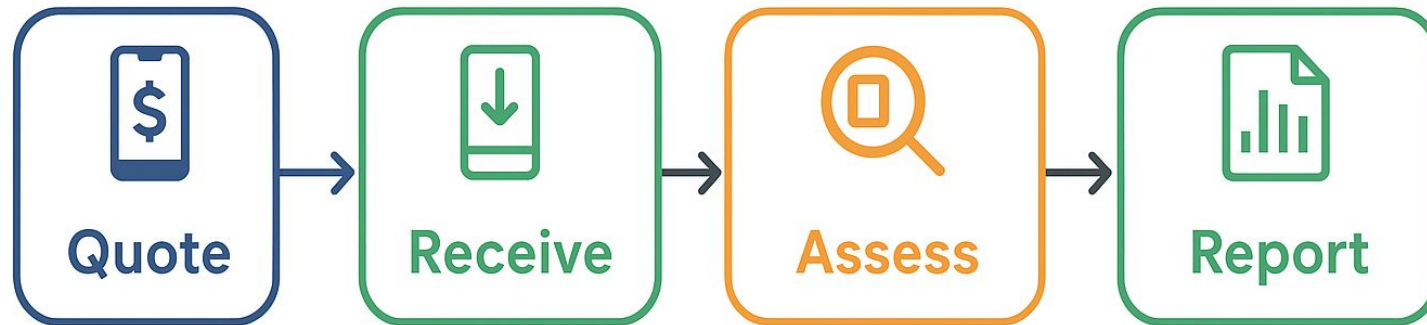


Figure 13-4. Adding an API layer between the user interface and domain services

Service-Based Architecture

- Sistema para reciclagem de Smartphones
 - Usuário requisita um orçamento usando o website ou uma máquina de autoatendimento
 - O processo de *quoting* irá definir quanto vale o smartphone do usuário
 - Se usuário aceitar a proposta de valor a receber, o usuário deve enviar o smartphone para a empresa (processo de *receiving*)
 - Ao receber o smartphone, a empresa irá decidir se está operável ou se será descartado (processo de *assessment*).
 - Se o dispositivo está como esperado, a empresa paga ao usuário (processo de *accounting*).
 - O usuário pode ir ao website da empresa a qualquer momento e verificar o status da reciclagem (processo de *item status*).
 - A partir do resultado do processo de *assessment*, o dispositivo é destruído ou revendido (processo de *recycling*).
 - Por fim, a empresa executa periodicamente a geração de relatórios para as atividades de reciclagem (processo de *reporting*).

Service-Based Architecture



Service-Based Architecture

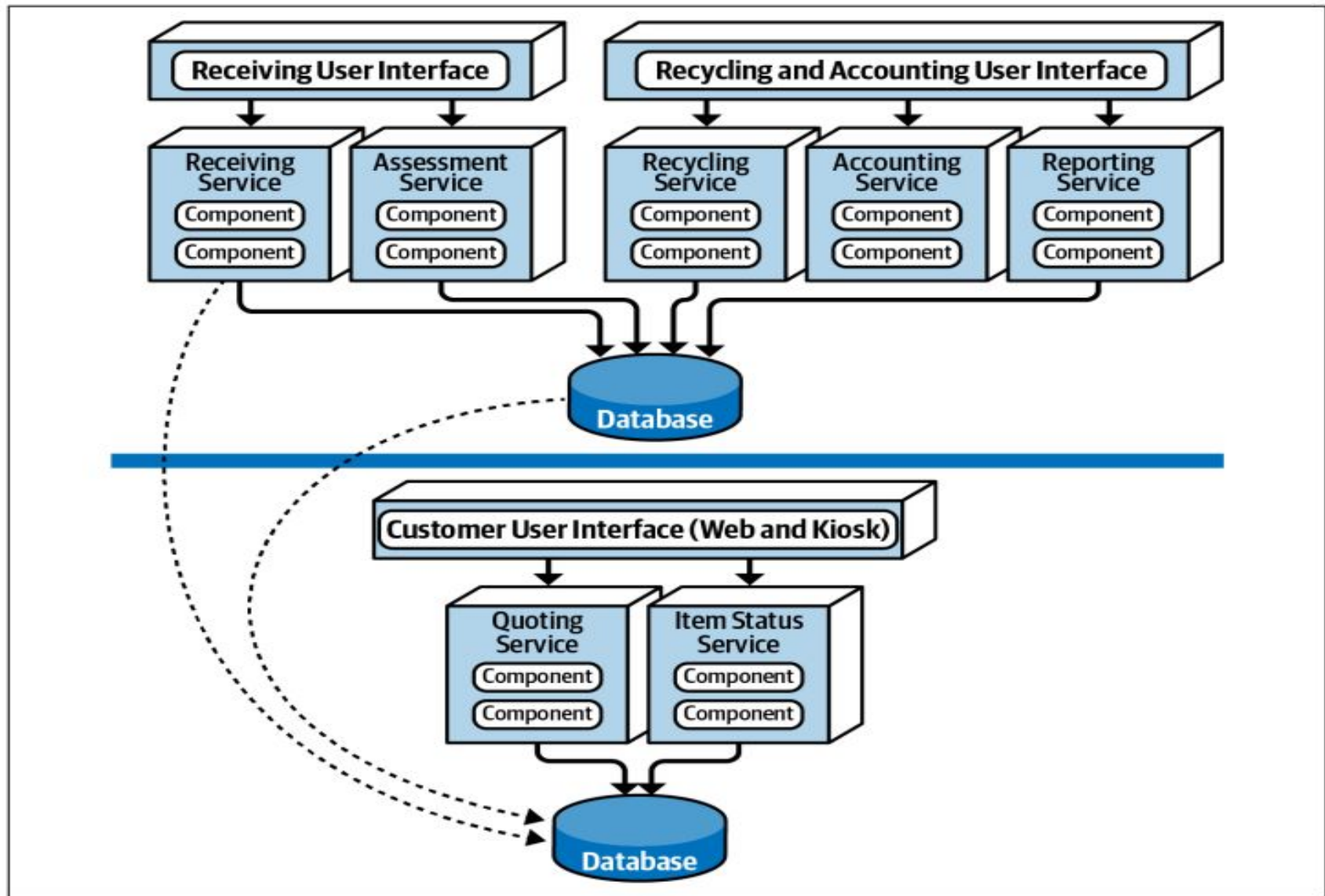


Figure 13-8. Electronics recycling example using service-based architecture

Service-Based Architecture

- Atente a separação de cada “domínio” como um módulo / serviço do sistema.
- Alguns módulos / serviços podem ser escalados de acordo com a necessidade, sem precisar fazer o *deploy* de todo o sistema.
 - Como nas arquiteturas monolíticas...

Service-Based Architecture

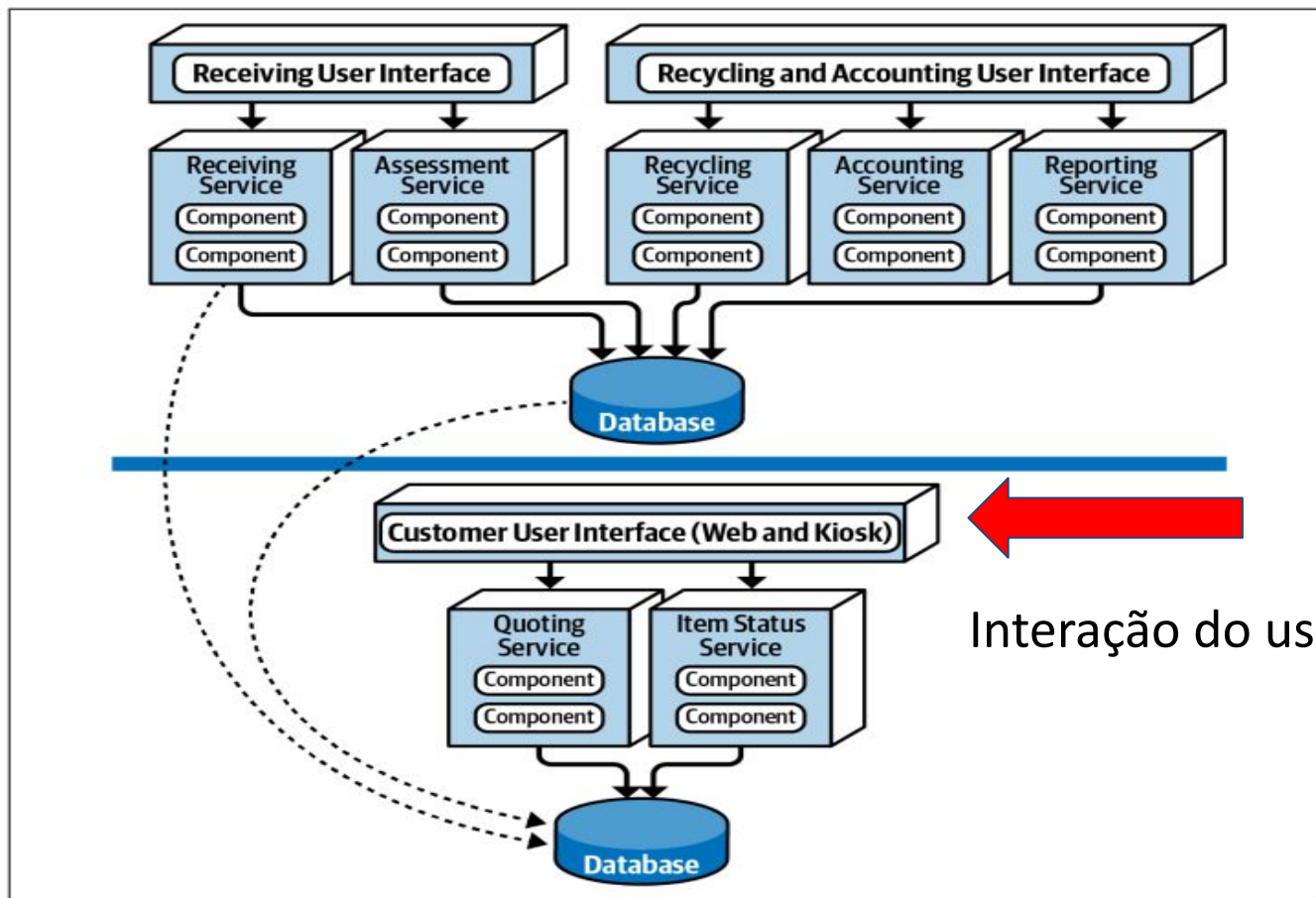


Figure 13-8. Electronics recycling example using service-based architecture

O usuário não precisa conhecer a interface dos demais serviços (internos).
Quais atributos de qualidade podem ser impactados por essa decisão?

Service-Based Architecture

- Escalabilidade
 - É possível escalar serviços distintos
- Tolerância a falhas
 - Se um serviço falhar é mais fácil reparar ou substituir
- Segurança
 - Usuário não conhece procedimentos internos dos serviços
- Processo de *assessment* é alterado constantemente para adicionar novas regras de negócios e novos produtos que podem ser recebidos.
- Essa alteração frequente é isolada do restante do sistema, provendo
 - Agilidade, ou seja, resposta rápida as mudanças
 - Testabilidade, ou seja, é mais fácil testar
 - *Deployability*, ou seja, reduzindo os riscos associados a um *deploy*. Se é implantado somente um serviço é mais simples e menos perigoso.

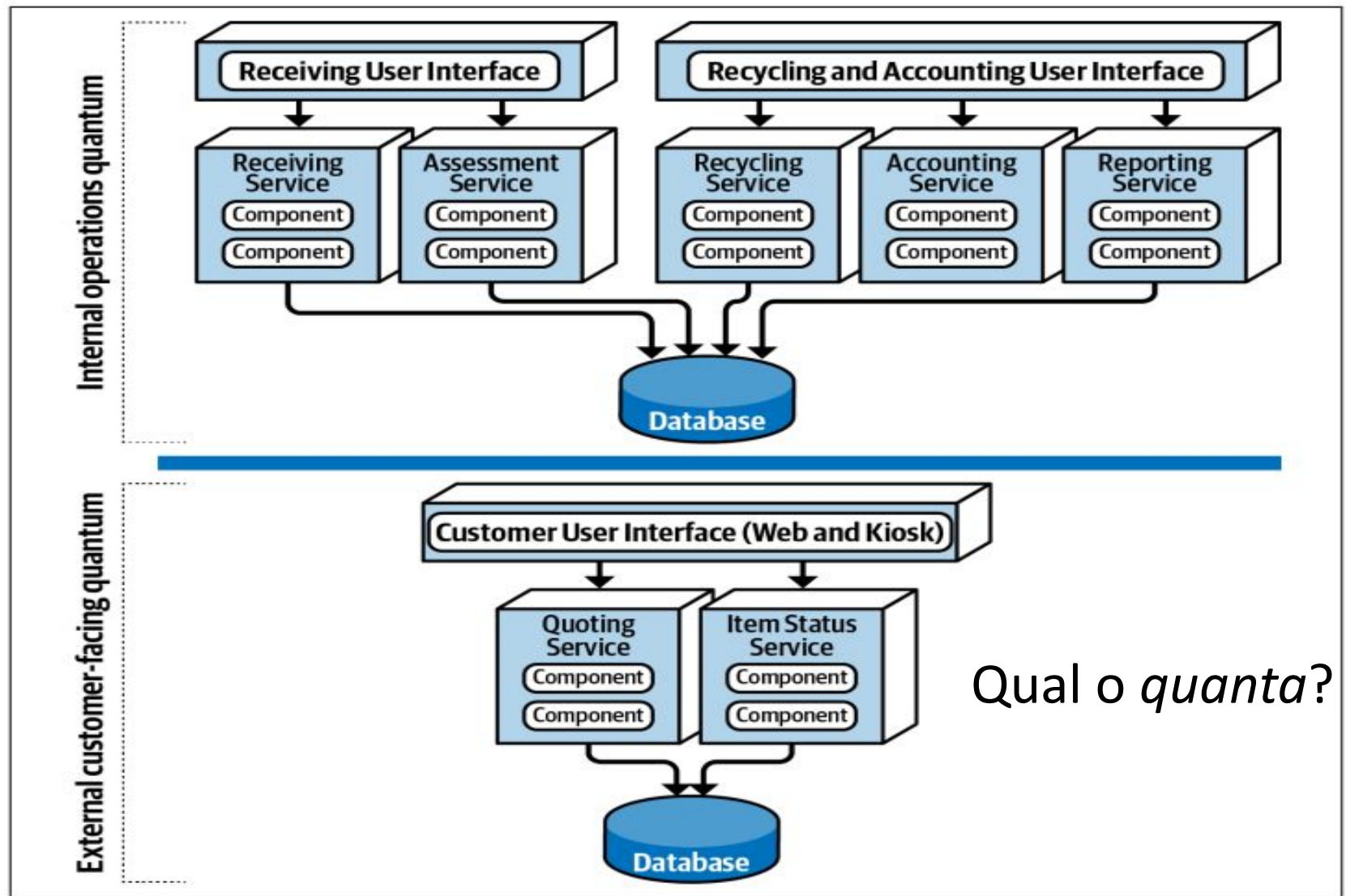
Service-Based Architecture

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

Service-Based Architecture

- Como uma arquitetura distribuída, o *quanta* pode ser maior ou igual a 1.
- Porém, se houver 10 ou mais serviços, mas que compartilham o mesmo banco de dados, não haverá “elementos isolados” e o *quanta* será 1
 - *Quanta* é a menor unidade lógica ou funcional de um sistema que pode ser escalada ou distribuída independentemente
- A seguir um exemplo de outra modelagem para o sistema em análise...

Service-Based Architecture



Qual o *quanta*?

Figure 13-10. Separate quanta in a service-based architecture

Service-Based Architecture

- Onde podemos aplicar a arquitetura baseada em serviços?
- Cite alguns exemplos de aplicações que podem se beneficiar das características dessa arquitetura.

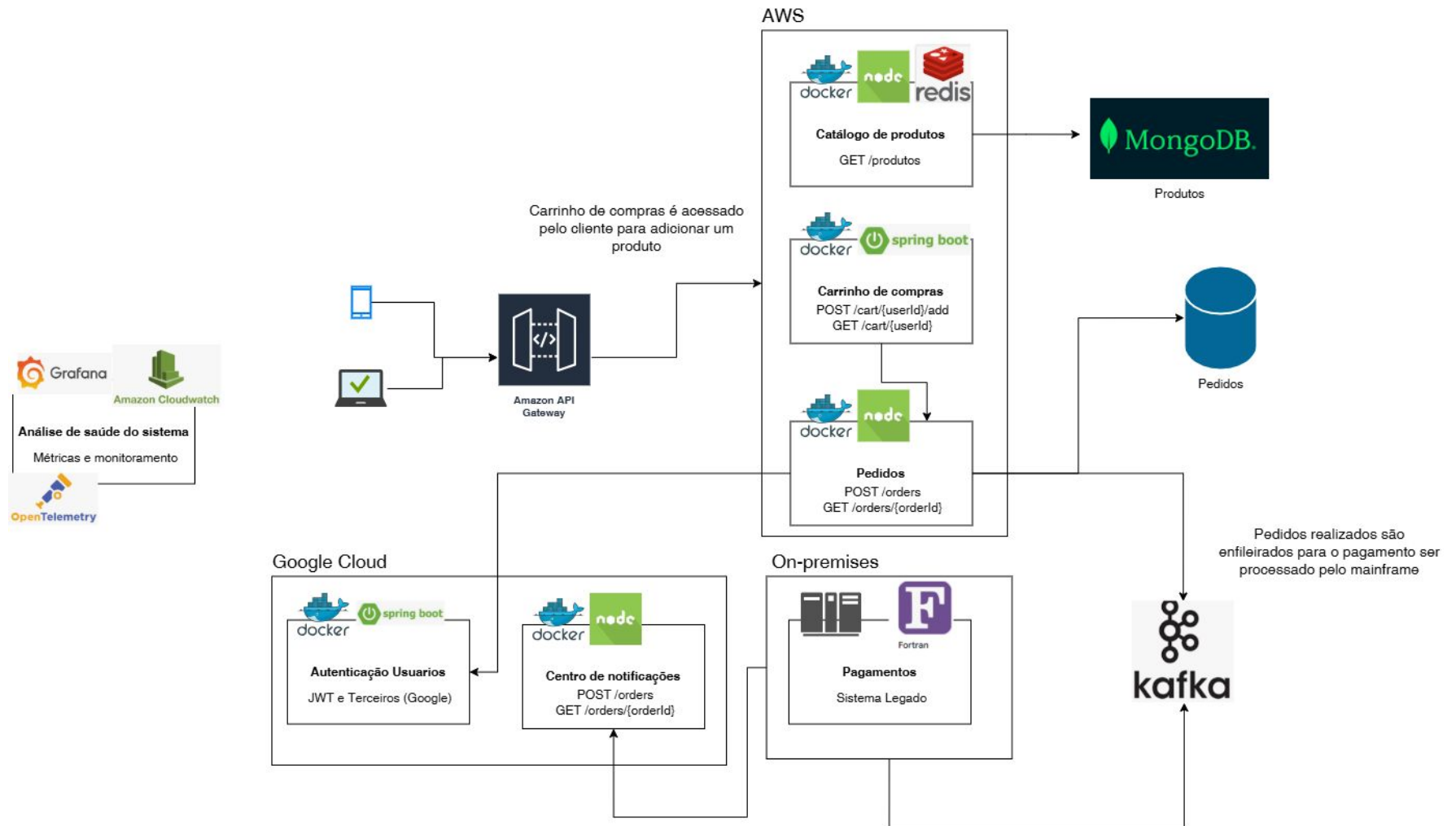
Service-Based Architecture

- A arquitetura baseada em serviços tem um bom balanceamento entre quase todas as características, sendo uma boa escolha para diversos nichos de aplicações.
- Porém, cuidado ao escolher essa arquitetura e o custo para criar e manter ser menor do que o benefício proporcionado.
- Cada domínio do sistema pode ser associado facilmente a um serviço, tornando essa arquitetura uma boa escolha para aplicações divididas por domínio.

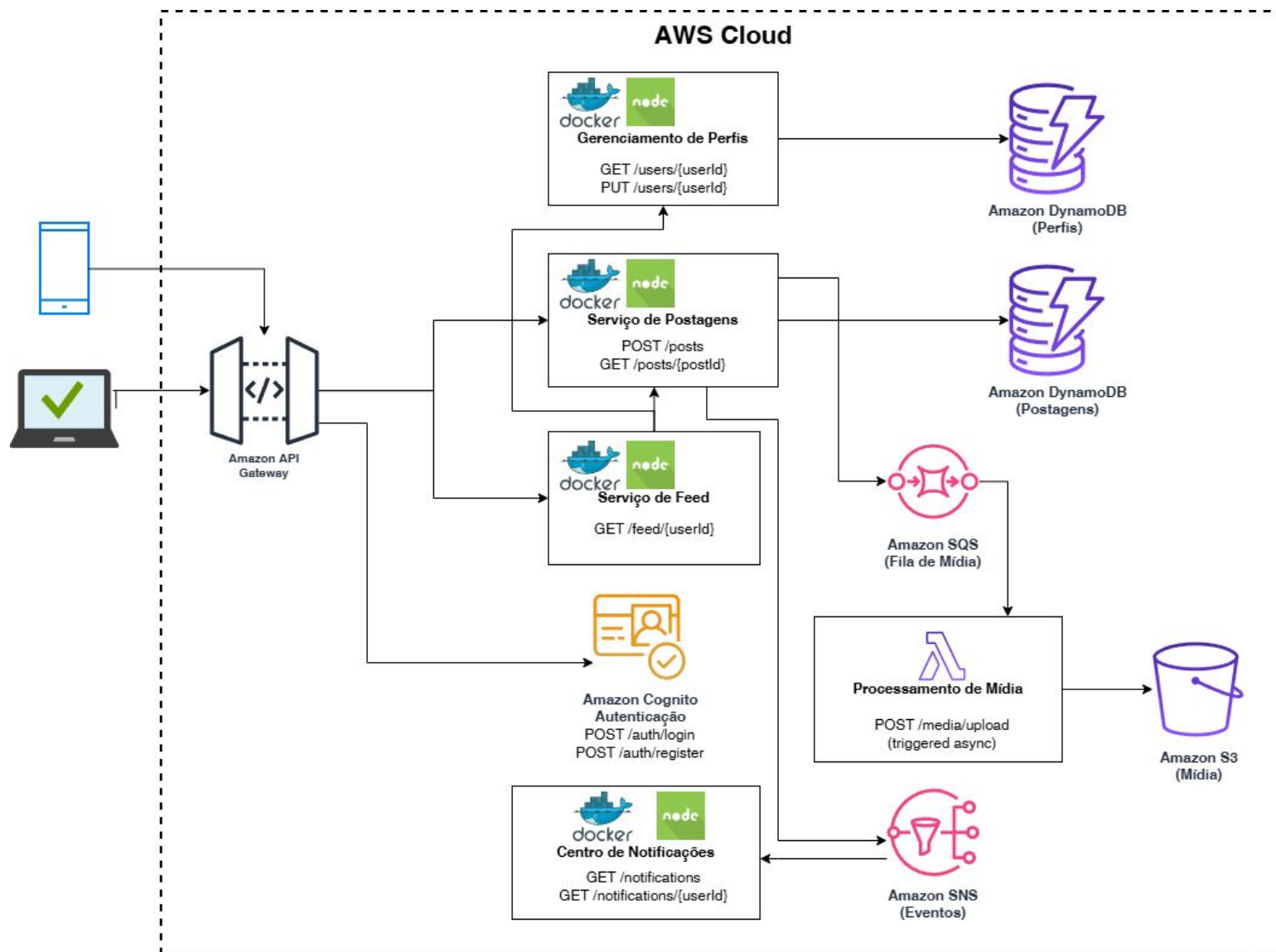
Service-Based Architecture

- Por fim, essa arquitetura não traz a granularidade fina dos microserviços (dezenas/centenas de serviços) e alcança uma modularidade adequada para a maioria das aplicações.
- Porém, se o seu domínio pode ser quebrado em muitos serviços e exige uma granularidade realmente fina, a arquitetura de microserviços pode ser uma alternativa mais adequada.

Service-Based Architecture



Service-Based Architecture



Dúvidas?



www.shutterstock.com · 744867163

ATIVIDADE

- Construa um exemplo de arquitetura para o tema do seminário da sua equipe ou outro sistema de sua escolha.
- É uma atividade individual, embora seja o mesmo tema, deve haver diferenças significativas entre a dos coleguinhas de equipe.
- Você deve utilizar a ferramenta web draw.io para criar os elementos da arquitetura.
 - 1 modelagem usando os princípios da arquitetura hexagonal;
 - 1 modelagem usando os princípios da arquitetura microkernel;
- Lembre que quanto mais específico for, melhor poderá ser a nota. Cuidado para não ser muito abstrato.
- Para auxiliar nesse processo, tente criar a modelagem a nível de módulos, onde cada módulo pode ser responsável por uma ou mais funcionalidades do sistema.
- Use ilustrações (imagens ou ícones do draw.io) para servidores, balanceadores de cargas, APIs, bancos de dados, etc.

Event-Driven Architecture

Event-Driven Architecture

- Estilo arquitetural distribuído muito popular usado para pequenas aplicações, altamente escaláveis e de alto desempenho.
- Baseada no desacoplamento de componentes a partir de eventos, que produzem dados de forma assíncrona e outro componente é responsável por processar estes dados.
- Pode ser usado de forma única ou em conjunto com outras arquiteturas, por exemplo, a Hexagonal, Microkernel ou Microserviços.
- A maioria das aplicações segue um padrão do “produtor-consumidor” onde requisições dos clientes são enviadas a um servidor para orquestração, cada requisição pode ser redirecionada para um outro servidor especializado ou uma instância de “consumidor”.

Event-Driven Architecture

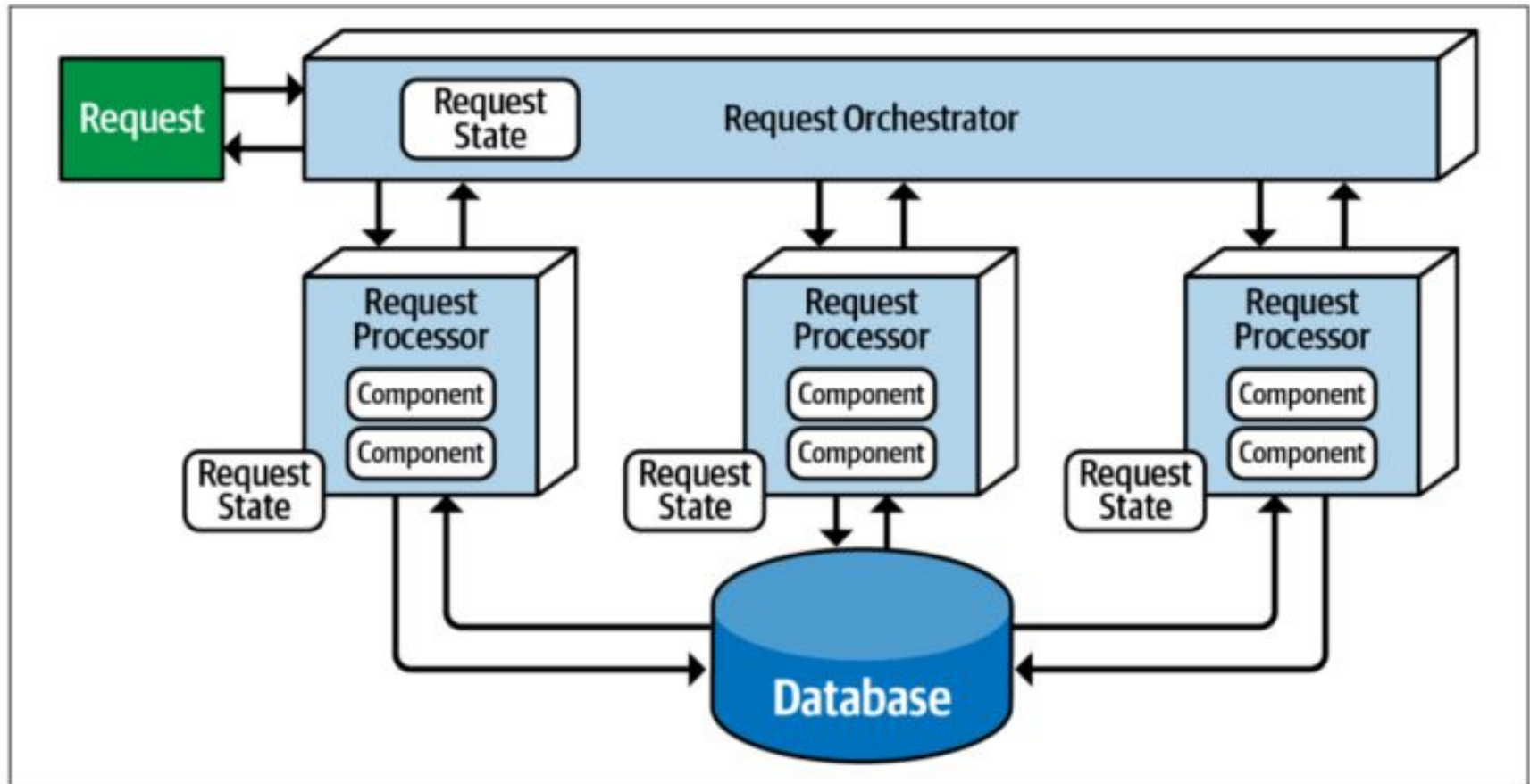


Figure 14-1. Request-based model

Uma pequena pausa para “Event-Driven Architecture + Hexagonal” Exemplo prático

Event-Driven Architecture + Hexagonal

- Vamos modelar um **sistema de gerenciamento de pedidos** virtual
 - Os pedidos podem ser criados, atualizados e cancelados.
 - Integração com um sistema de pagamento externo.
 - Integração com uma plataforma de envio (logística).
 - Persistência de dados em um banco de dados.
 - Eventos disparam ações como envio de notificações por e-mail, SMS e Whatsapp.

Event-Driven Architecture + Hexagonal

- Núcleo Hexagonal (Domínio)
 - Contém toda a lógica de negócio relacionada aos pedidos.
- Portas e Adaptadores
 - Facilitam a comunicação entre o núcleo e o mundo externo.
- Eventos (Event-Driven)
 - Utilizados para integrar módulos internos e sistemas externos de forma desacoplada.

Event-Driven Architecture + Hexagonal

- Componentes Chave:
 - Domínio do Pedido (Hexagonal): Responsável por gerenciar pedidos.
 - Message Broker: Gerencia a publicação e o consumo de eventos.
 - Sistemas Externos: Como sistemas de pagamento e logística.
 - Banco de Dados: Para persistência dos pedidos e eventos.
 - Notificações: Para informar clientes sobre mudanças no status do pedido.

Event-Driven Architecture + Hexagonal

- Núcleo Hexagonal (Domínio)
 - Regras de negócio centralizadas no domínio do pedido.
 - Manipulação de entidades e serviços de negócio.
 - Define portas (interfaces) para comunicação com o exterior.
- Principais Portas:
 - Portas de Entrada (Input Ports)
 - CriarPedidoService
 - AtualizarPedidoService
 - CancelarPedidoService
 - Portas de Saída (Output Ports):
 - PedidoRepositoryInterface
 - Para persistência dos dados.
 - PagamentoGatewayInterface
 - Para comunicação com o sistema de pagamento.
 - LogisticaServiceInterface
 - Para integração com a logística.
 - NotificationServiceInterface

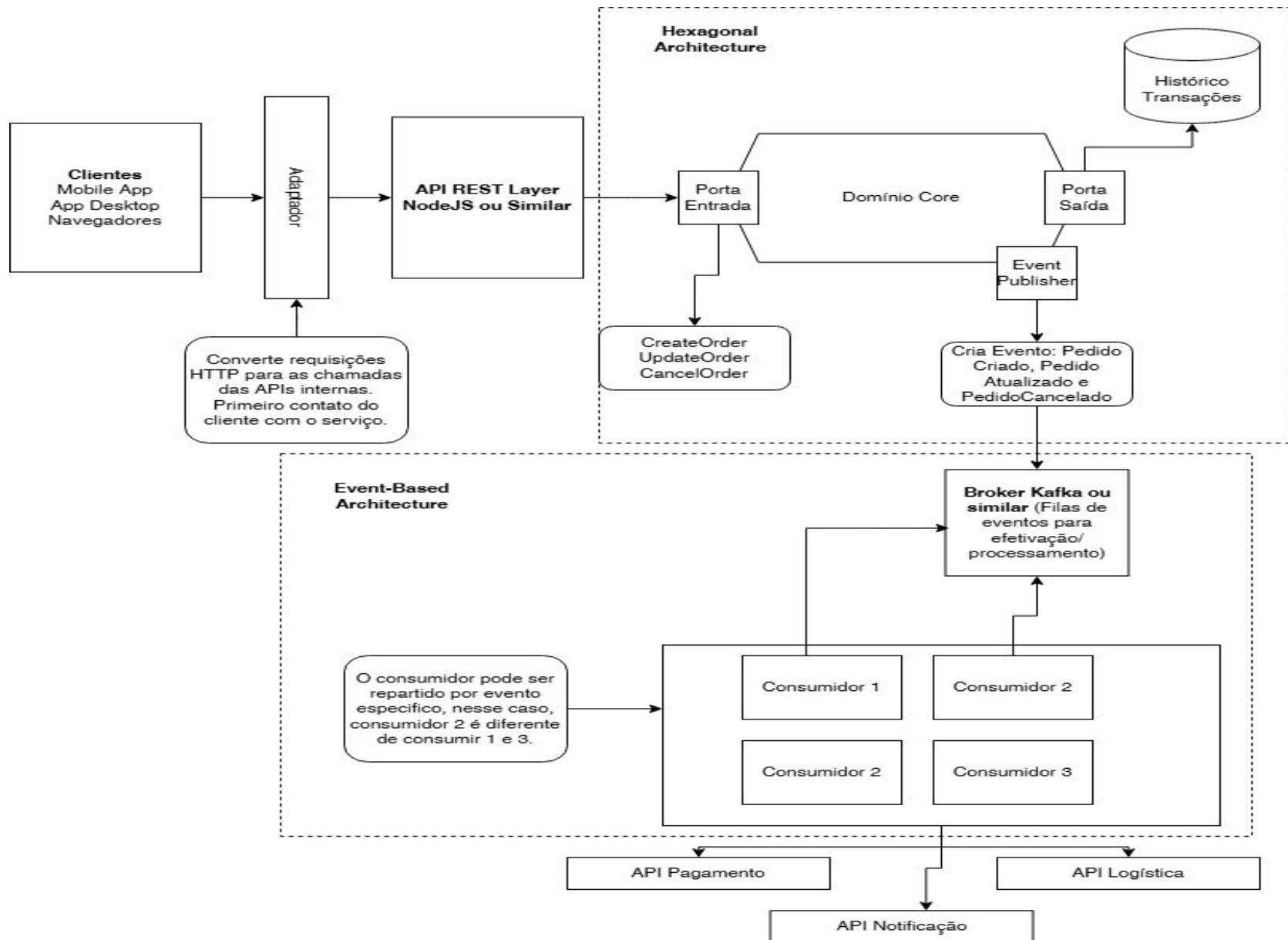
Event-Driven Architecture + Hexagonal

- Event-Driven Architecture
 - Message Broker: RabbitMQ ou Kafka
 - Centraliza a comunicação baseada em eventos entre módulos.
 - Permite que o sistema reaja a mudanças de forma desacoplada.
- Eventos Principais:
 - PedidoCriado: Disparado quando um pedido é criado.
 - PedidoAtualizado: Disparado quando um pedido é atualizado.
 - PedidoCancelado: Disparado quando um pedido é cancelado.
 - PagamentoConfirmado: Disparado quando o pagamento é concluído.

Event-Driven Architecture + Hexagonal

1. Um cliente cria um pedido através da API REST.
1. O núcleo valida e cria o pedido.
1. O evento PedidoCriado é publicado no Message Broker.
1. Os seguintes consumidores reagem ao evento (pode ser assíncrono):
 - a. Serviço de Pagamento: Inicia a cobrança.
 - b. Serviço de Notificação: Envia uma confirmação ao cliente.
 - c. Serviço de Logística: Prepara o processamento do envio.

Event-Driven Architecture + Hexagonal



Event-Driven Architecture + Hexagonal

- Benefícios da Combinação
 - Modularidade
 - A hexagonal organiza o sistema em módulos altamente desacoplados.
 - Escalabilidade
 - A baseada em eventos permite que novos consumidores e emissores de eventos sejam adicionados facilmente.
 - Manutenção
 - Alterar a infraestrutura ou os consumidores não afeta o núcleo.
 - Resiliência
 - Os eventos podem ser processados de forma assíncrona, reduzindo o impacto de falhas.
 - Escalabilidade
 - Mais consumidores podem ser criados para melhorar o fluxo e atender grande quantidade de pedidos.

Voltando para a Event-Driven Architecture somente...

Event-Driven Architecture

- Um modelo baseado em eventos reage a uma determinada situação e aplica uma ação aquele evento.
- Um exemplo é a submissão de uma oferta em um leilão online, onde o cliente faz um registro de oferta, mas essa oferta não é a efetivação da compra.
- O registro da oferta é um evento registrado no sistema que, após a conclusão do leilão serão executadas as ações de escolha do maior lance, efetivação da venda e processamento do pagamento.

Event-Driven Architecture

- A arquitetura baseada em eventos pode ser modelada de duas formas:
 - *Mediator topology*
 - *Broker topology*
- No *mediator* há uma coordenação central onde a requisição é quebrada e enviada para cada componente de forma coordenada, gerindo todo o processo e agregando as respostas.
 - Usada quando há muitos passos que precisam ser coordenados ou sequenciados. Se usamos com a Hexagonal, parece estar mais associada a um *mediator*.
- No *broker* há uma abordagem mais distribuída, onde o *broker* encaminha as requisições para módulos ou servidores que são responsáveis por processar a solicitação.

Event-Driven Architecture

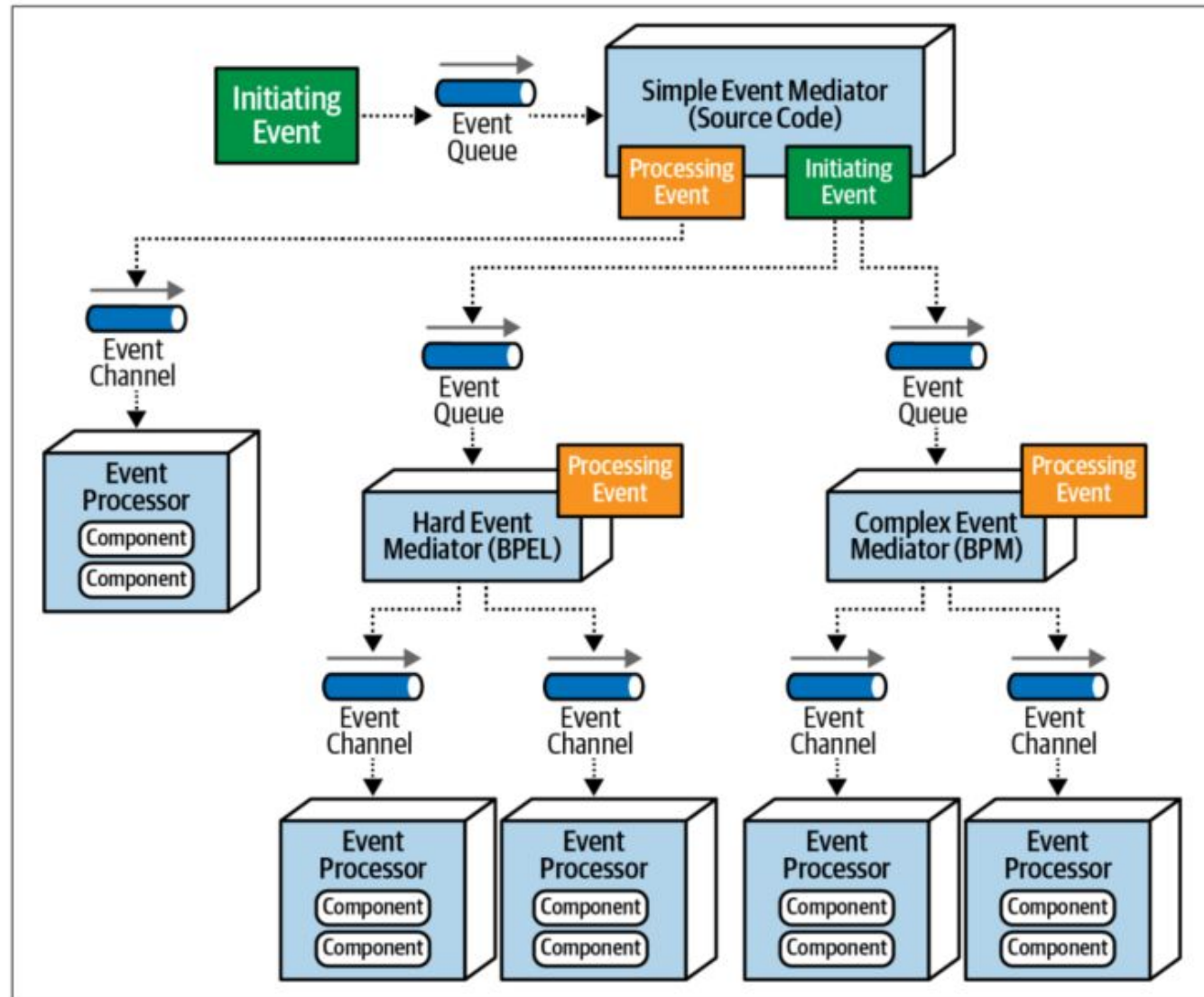


Figure 14-6. Delegating the event to the appropriate type of event mediator

Event-Driven Architecture

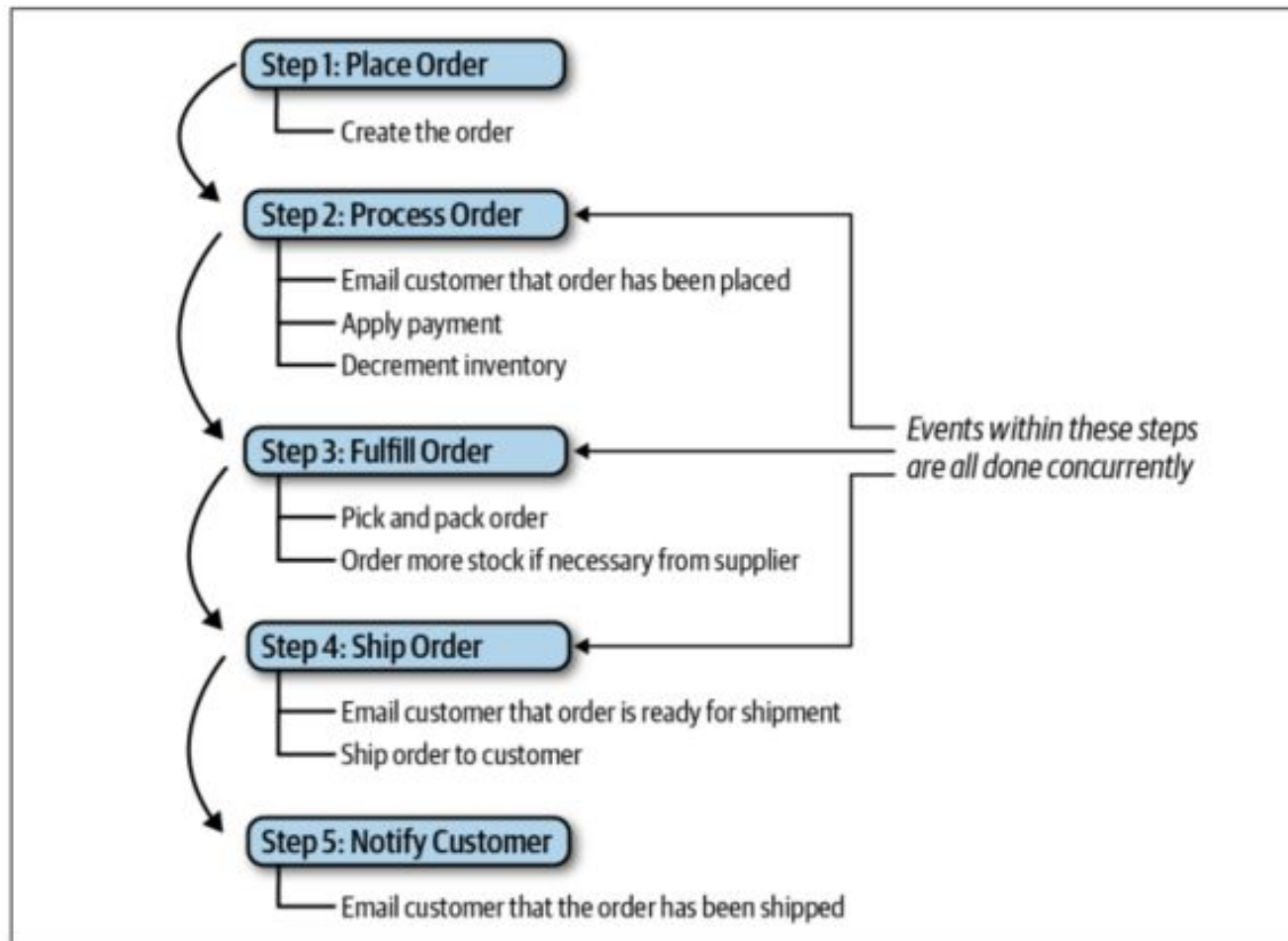
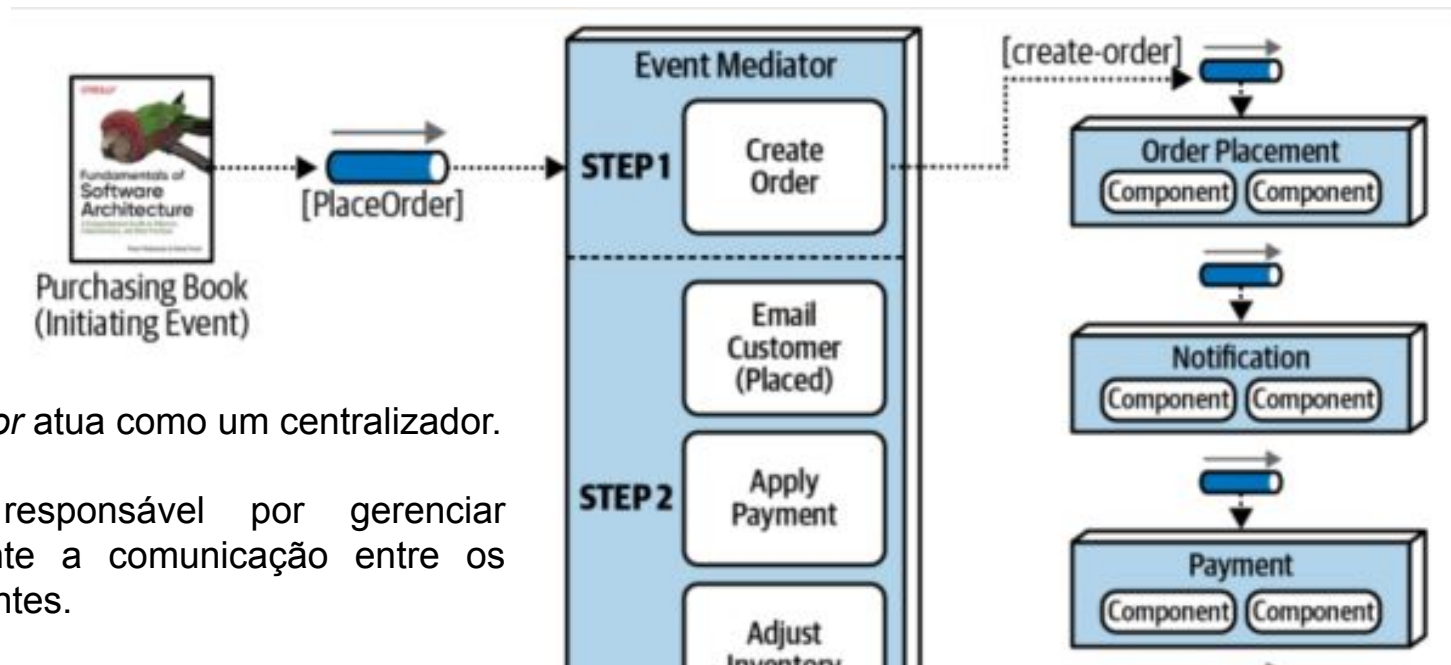


Figure 14-7. Mediator steps for placing an order

Event-Driven Architecture

- Sequência dos eventos e ações ao construir uma abordagem baseada no *mediator*



O *Mediator* atua como um centralizador.

Ele é responsável por gerenciar diretamente a comunicação entre os componentes.

Os emissores enviam eventos para o Mediator, que os direciona explicitamente aos receptores apropriados

Event-Driven Architecture

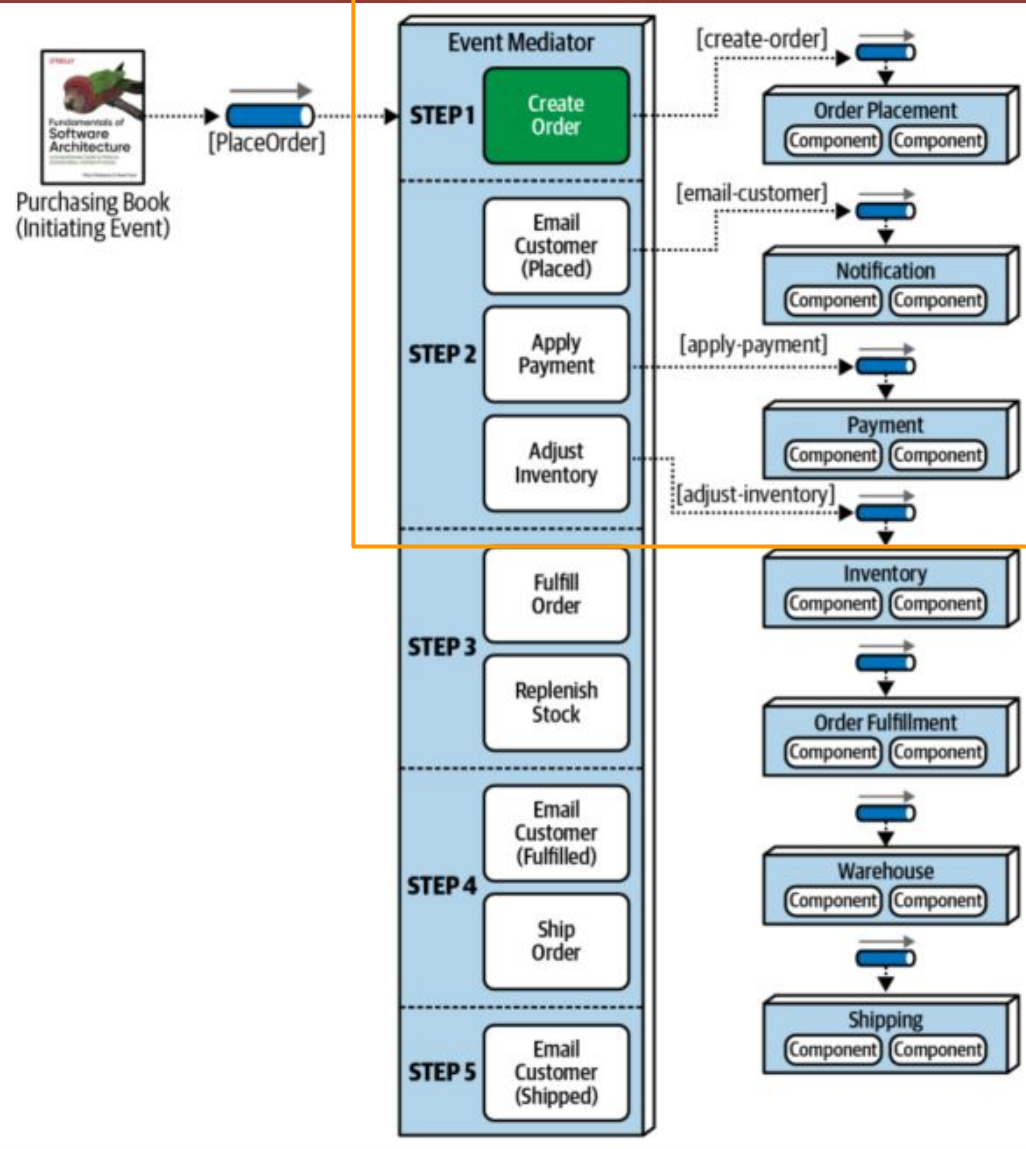
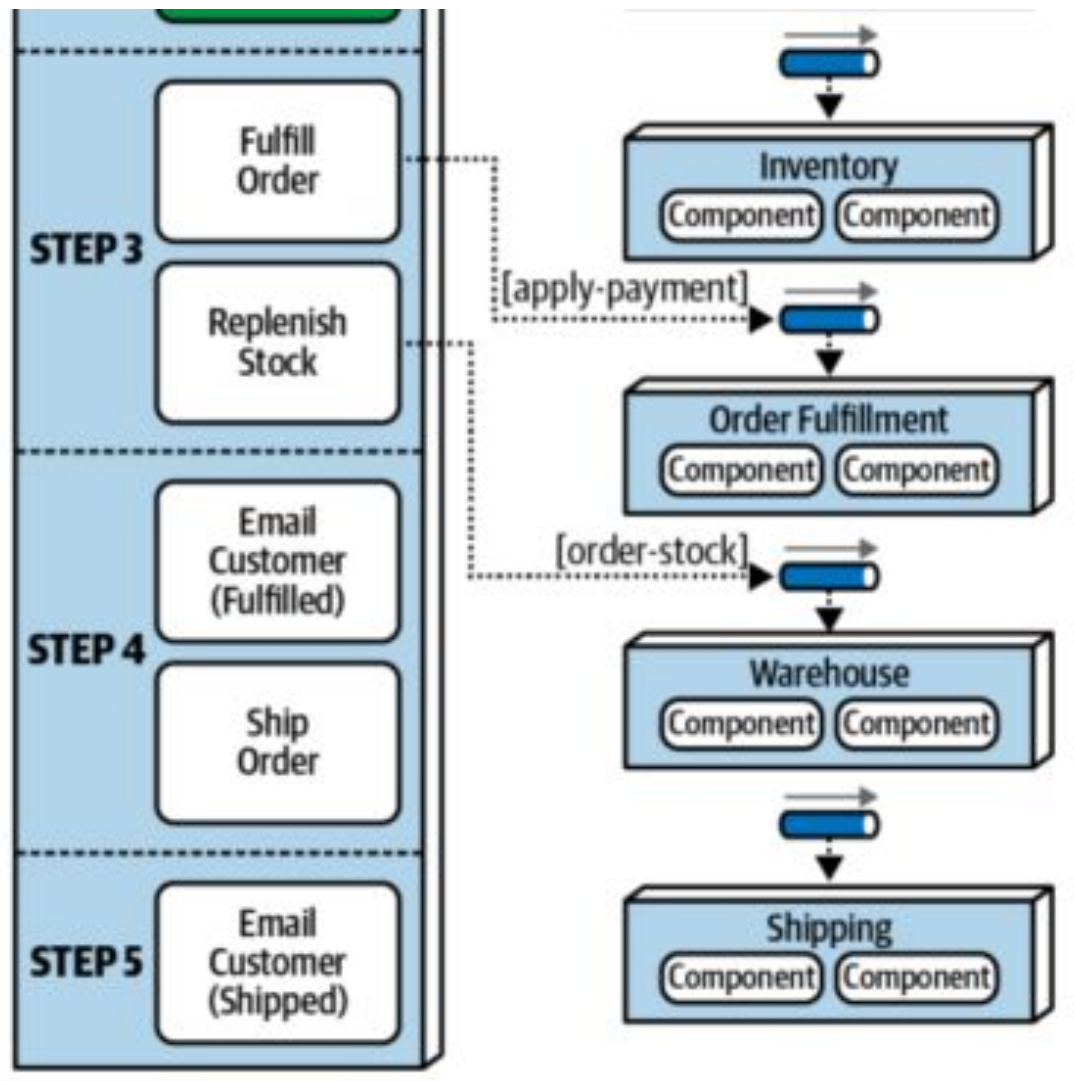
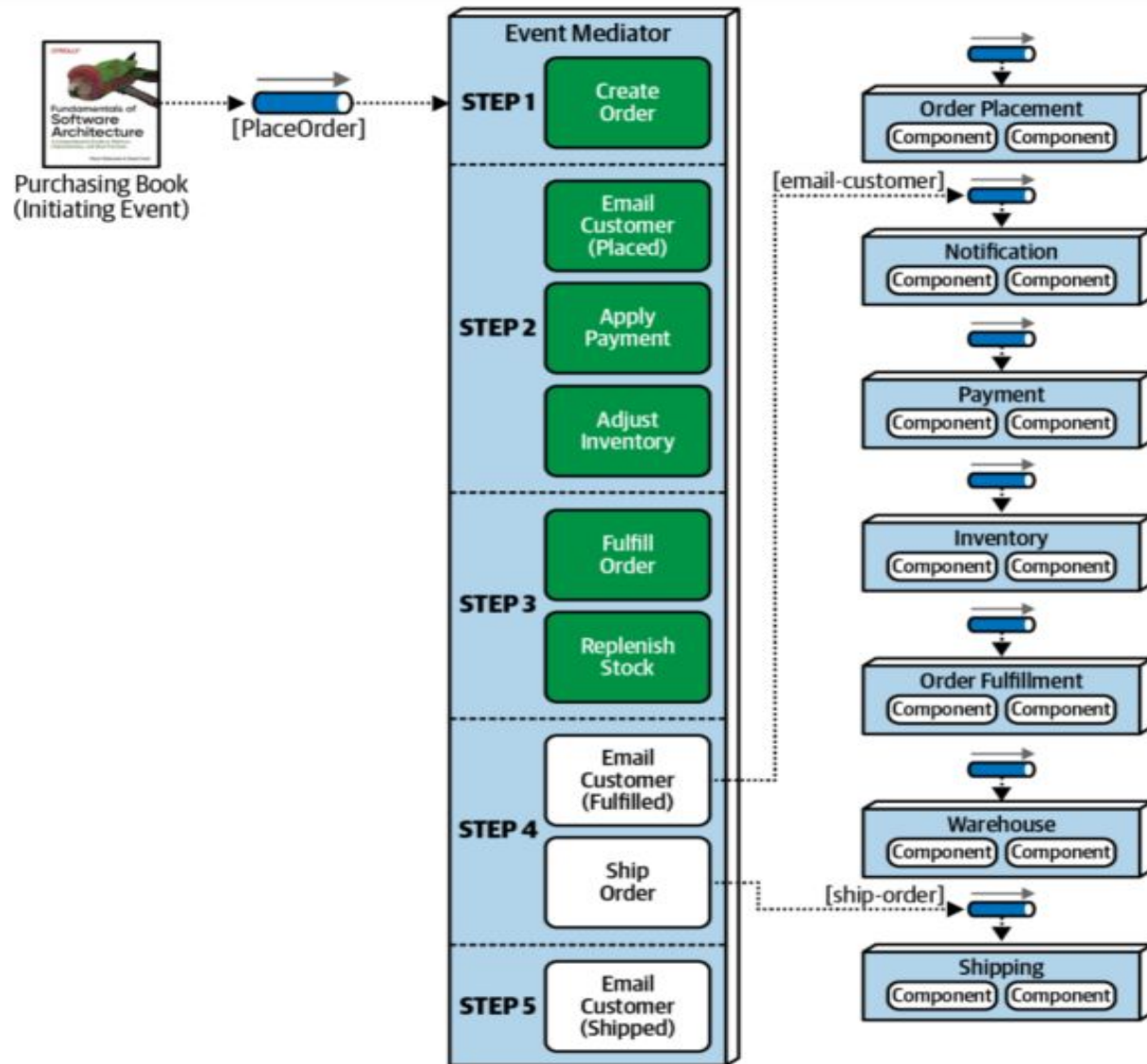


Figure 14-9. Step 2 of the mediator example

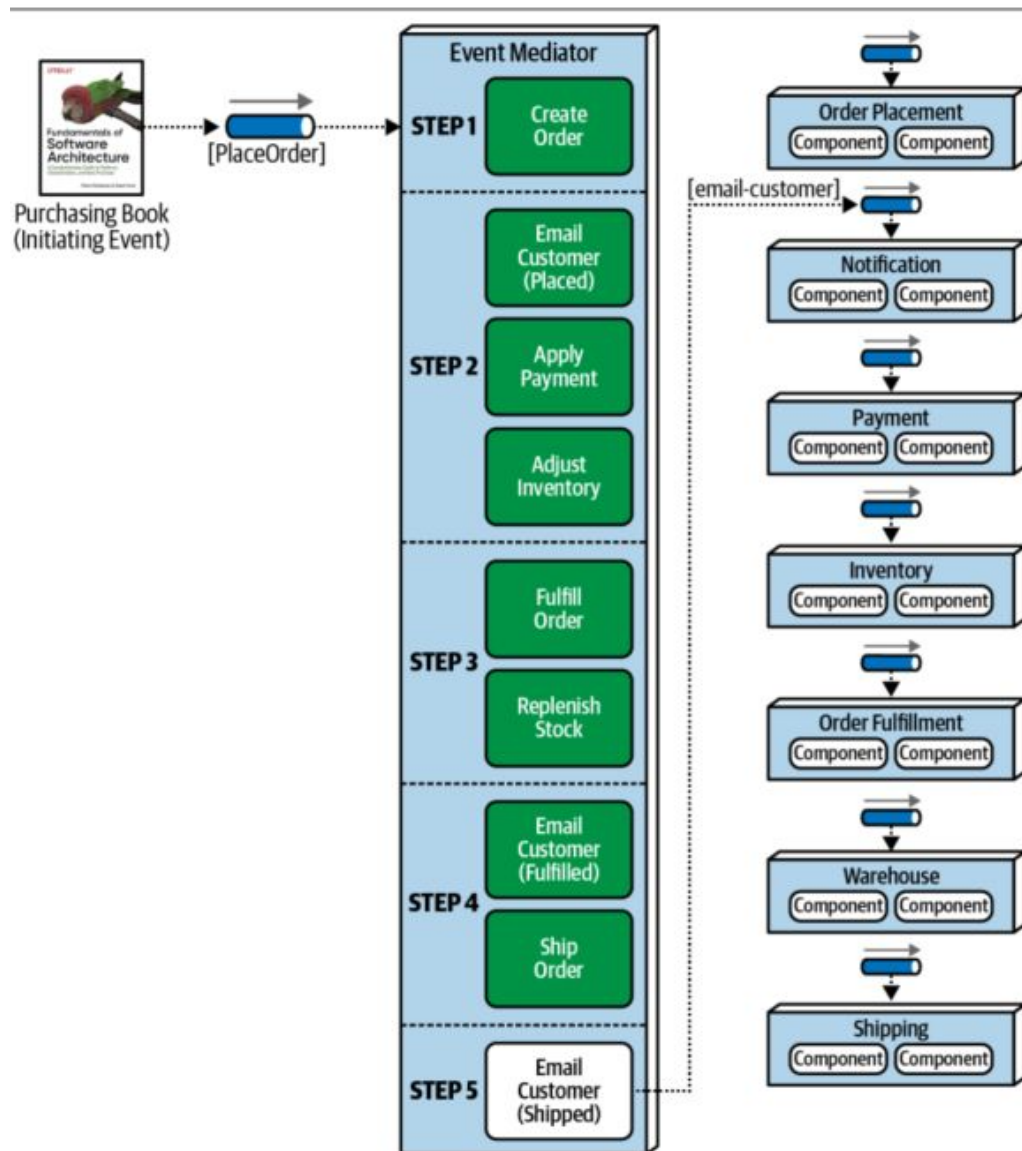
Event-Driven Architecture



Event-Driven Architecture



Event-Driven Architecture



Event-Driven Architecture

Broker Example

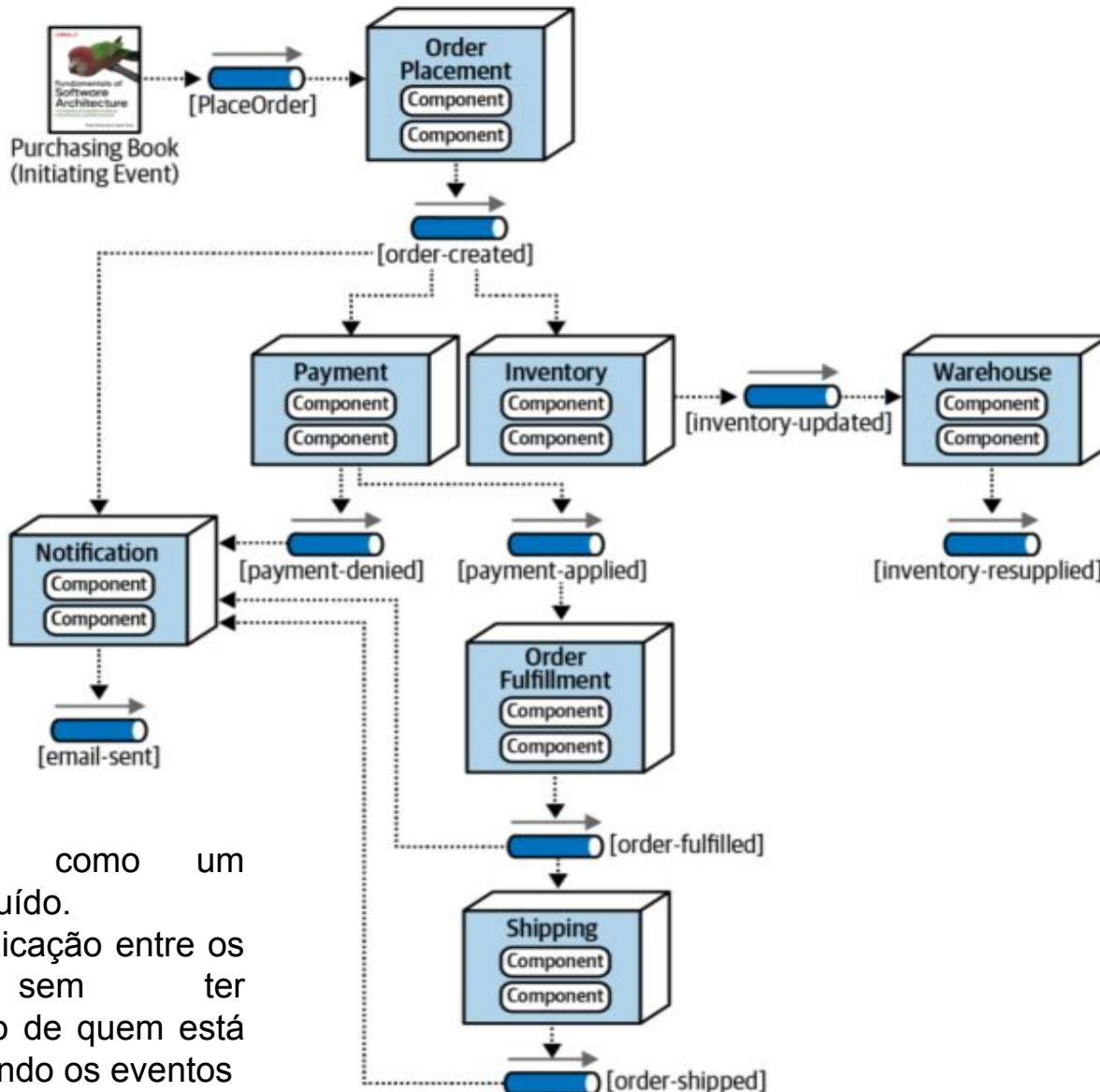


Figure 14-4. Example of the broker topology

O Broker atua como um intermediário distribuído. Ele facilita a comunicação entre os componentes sem ter conhecimento direto de quem está enviando ou recebendo os eventos

Event-Driven Architecture

- É perceptível que por ser uma comunicação assíncrona, a perda de dados ou requisições deve ser tratada de forma mais protetiva, evitando a perda de requisições ou dados durante o processo de comunicação dos módulos ou servidores.
- Por exemplo, a requisição pode ser
 - parcialmente executada no servidor;
 - haver uma falha na validação dos dados;
 - modelo do banco de dados rejeitar a transação;
 - dados serem perdidos no roteamento na internet; etc...

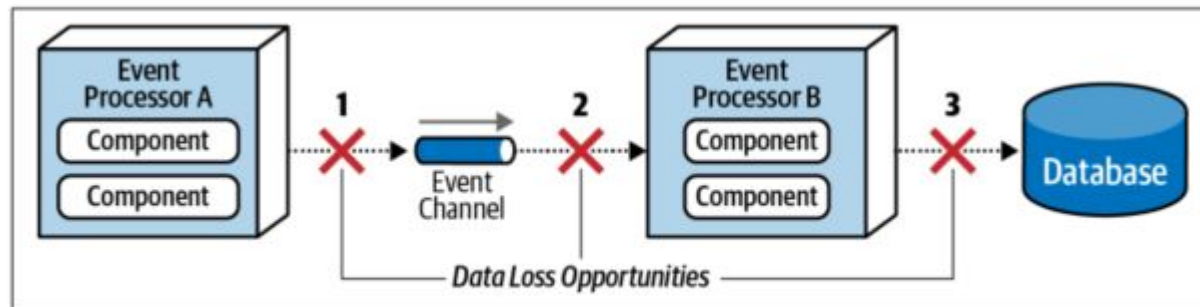


Figure 14-16. Where data loss can happen within an event-driven architecture

Event-Driven Architecture

- Cada evento que pode causar prejuízo aos dados ou transações possui técnicas específicas que podem ser agregadas para reduzir o risco.
- Por exemplo, usar filas persistentes (o evento somente é removido da fila após ser totalmente efetivado por todos os servidores).

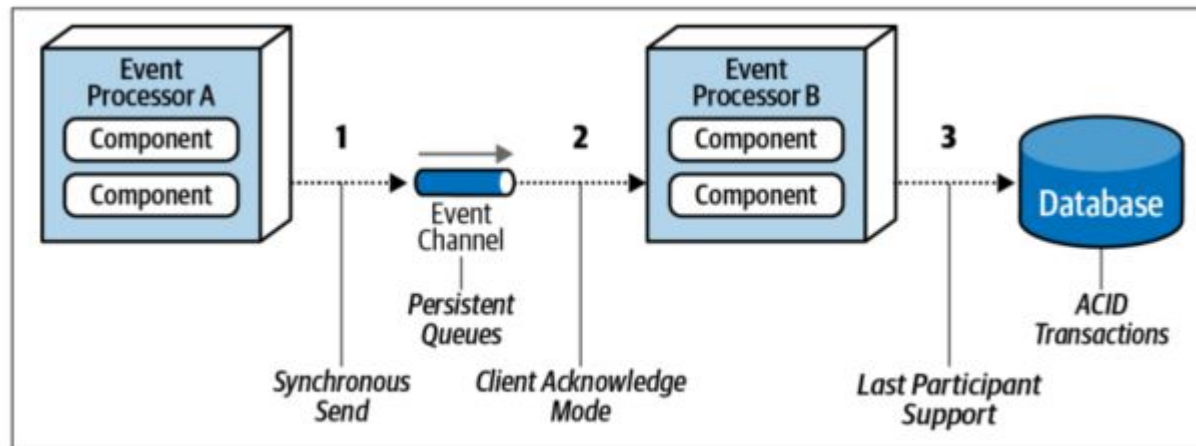


Figure 14-17. Preventing data loss within an event-driven architecture

Event-Driven Architecture

- Outra técnica que pode ser utilizada é o **broadcast** do evento para todos os módulos e servidores, onde cada um pode decidir o que fazer com aquele evento.
- Se não for “para mim” eu descarto o evento e volto a escutar a rede.
- Se houver mais de uma instância do mesmo servidor ou módulo executando pode haver uma validação no momento da efetivação da transação.
- Quem terminar primeiro registra a finalização, as propriedades do Banco de Dados garantem que não haja duplicação.
 - Infelizmente haverá trabalho “em vão”, pois um servidor processou a requisição e o outro também.

Event-Driven Architecture

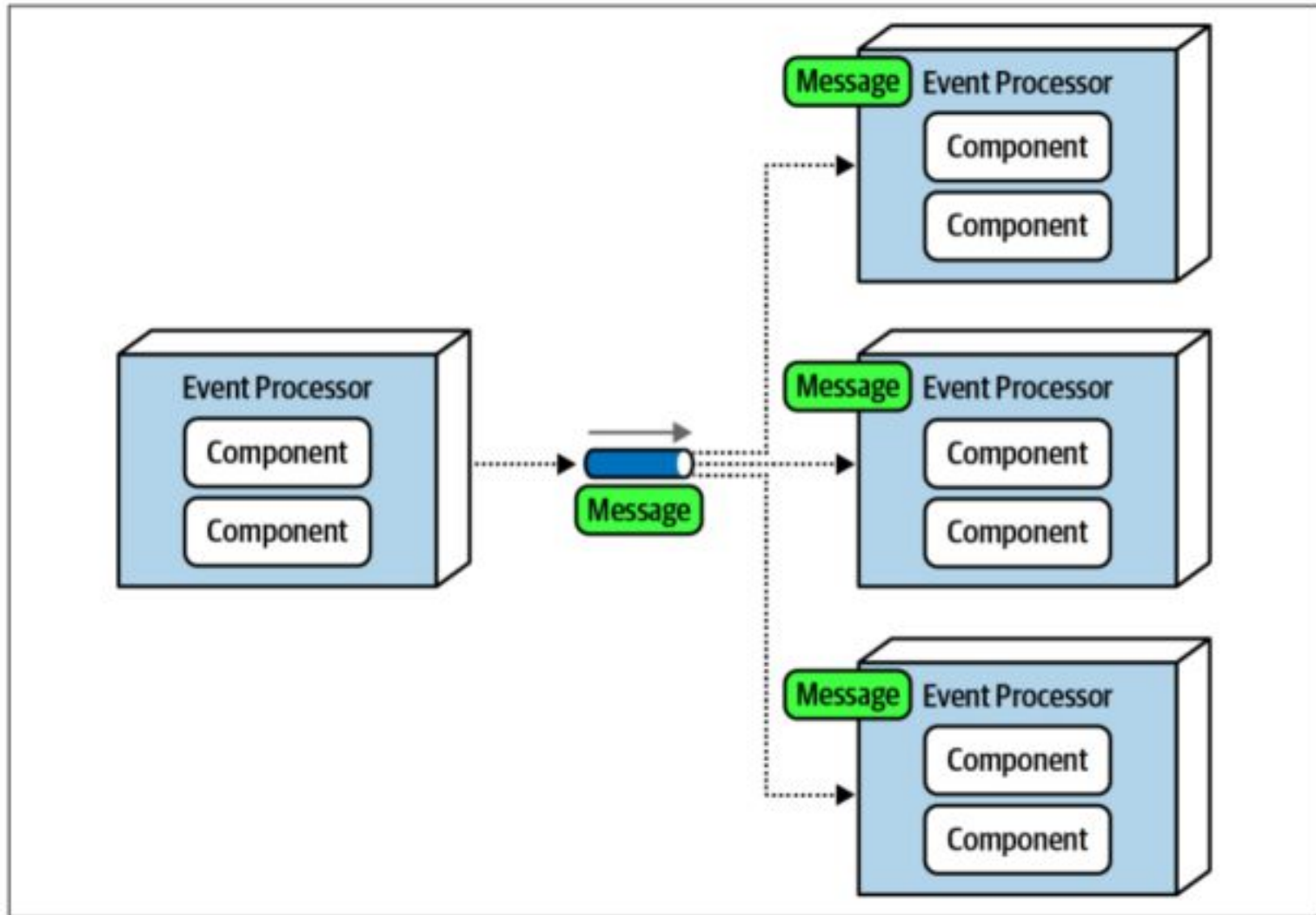


Figure 14-18. Broadcasting events to other event processors

Event-Driven Architecture

- Outra técnica que pode ser utilizada para prevenção de perda dos dados é a **mensagem de reconhecimento**.
- Parece reduzir os benefícios da abordagem distribuída?

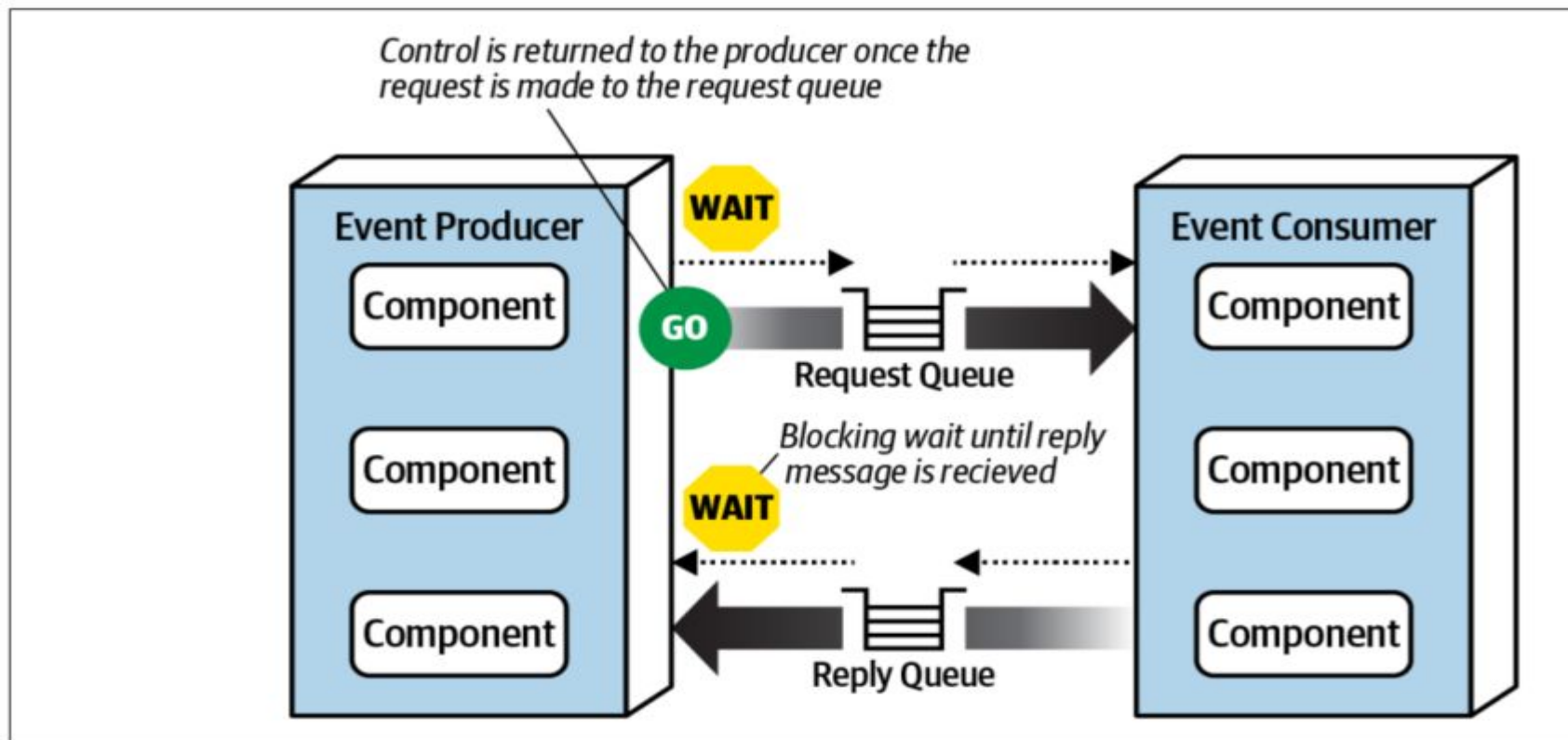


Figure 14-19. Request-reply message processing

Event-Driven Architecture

- Uma forma de implementar esta técnica é utilizando IDs na requisição e nas respostas, podendo associar que evento XY foi processado com sucesso.
 - CID = Correlation ID
 - O ID agora é o ID do evento processador.

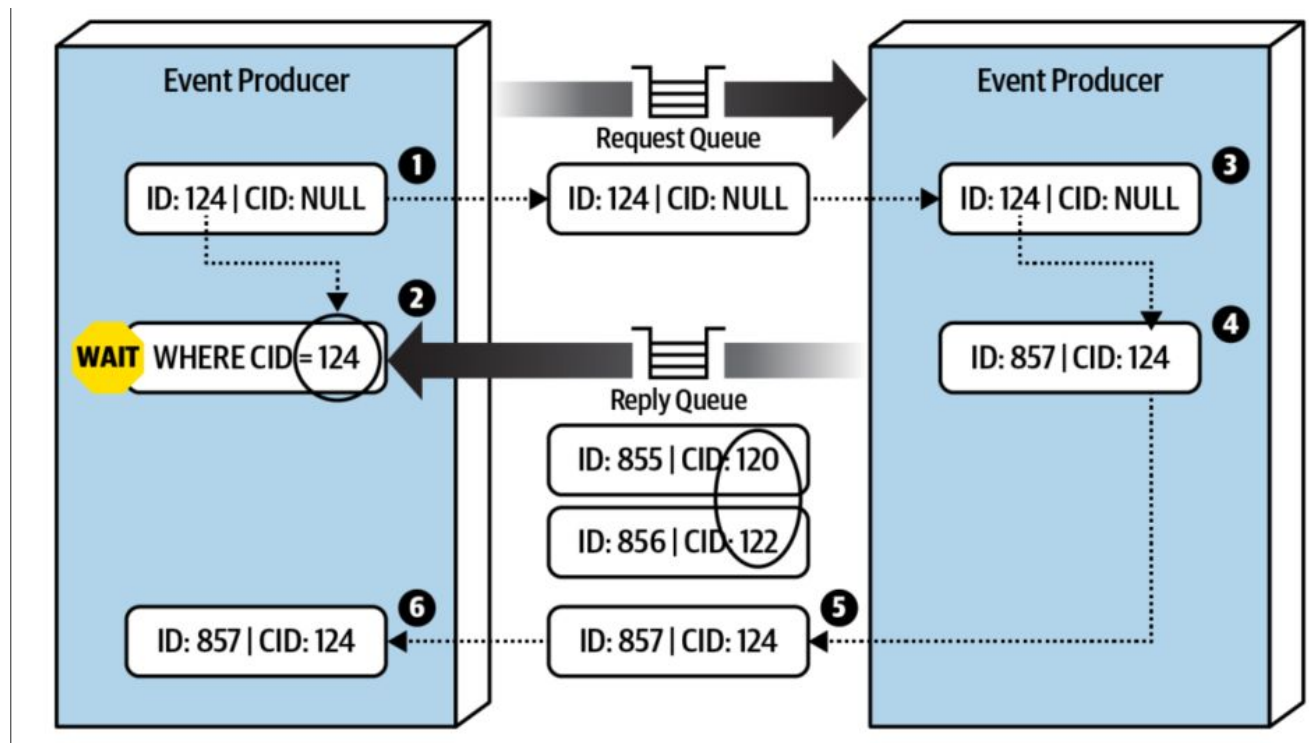


Figure 14-20. Request-reply message processing using a correlation ID

Event-Driven Architecture

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★
Evolutionary	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Modularity	★ ★ ★ ★
Overall cost	★ ★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★ ★

Event-Driven Architecture

- A quantidade de quanta na arquitetura baseada em eventos pode variar de 1 para muitos, dependendo principalmente da quantidade de bancos de dados, sendo que o serviço pode compartilhar o banco de dados.
- Mesmo usando principalmente requisições assíncronas, se o banco de dados é compartilhado, não temos uma unidade deployável independente (quanta = 1)
- Exemplo
 - Um evento envia uma requisição para outro processador para executar uma ordem de compra. O primeiro evento precisa esperar pelo id do pedido para continuar o processamento. Se o segundo evento estiver indisponível, o primeiro não conseguirá executar pela dependência, tornando-os um quanta único (quanta =1) mesmo se comunicando por mensagens assíncronas.

Event-Driven Architecture

- A arquitetura baseada em eventos apresenta 5 estrelas para desempenho, escalabilidade e tolerância a falhas, principais qualidades da arquitetura.
- Alto desempenho é alcançado pela comunicação assíncrona combinado com o processamento paralelizável.
- Alta escalabilidade é alcançada pelo balanceamento de eventos entre os computadores / serviços consumidores
 - Se a carga de processamento aumentar, é possível escalar novos consumidores.
- A tolerância a falhas é alcançada pelo alto desacoplamento dos componentes e os eventos assíncronos.

Event-Driven Architecture

- Por outro lado, é difícil realizar testes precisos, reduzindo as notas em simplicidade e testabilidade.
- Por possuir eventos assíncronos e não determinísticos, é difícil prever como um evento irá se comportar e ser integrado por outro evento.
- Por fim, a arquitetura baseada em eventos é altamente Evolucionária, é fácil adicionar funcionalidades adicionando novos processadores de eventos.
- Ademais, é preciso criar um novo formato do evento que será publicado em uma fila para ser lido por outro processador (consumidor)

<https://github.com/kudryashov/event-driven-architecture?tab=readme-ov-file>



Dúvidas?



www.shutterstock.com · 744867163

Service-based OR Event-Driven

- Crie uma arquitetura para uma loja online, por exemplo, Shopee ou Amazon (ou similar).
- Faça a modelagem usando arquitetura baseada em serviços ou orientada a eventos (ou as duas).
- Use o [Draw.IO](#) para criar o diagrama.
- Use ícones representativos das tecnologias, por exemplo, PostgreSQL, AWS S3, AWS EC2, REDIS, Docker, etc.
- Envio pelo SIGA.

Big Data Architectures

<https://learn.microsoft.com/pt-br/azure/architecture/databases/guide/big-data-architectures>

Big Data Architectures

- Arquitetura de Big Data é projetada para lidar com grandes massas de dados, envolvendo: ingestão e processamento dos dados, análise dos dados e persistência dos resultados.
- Para algumas organizações, Big Data pode significar centenas de gigabytes de dados, enquanto para outras, centenas de terabytes.
- Cada vez mais, esse termo se relaciona ao valor que é possível extrair dos conjuntos de dados por meio de análise avançada, em vez de estritamente o tamanho dos dados, embora nesses casos, eles tendem a ser muito grandes.

Big Data Architectures

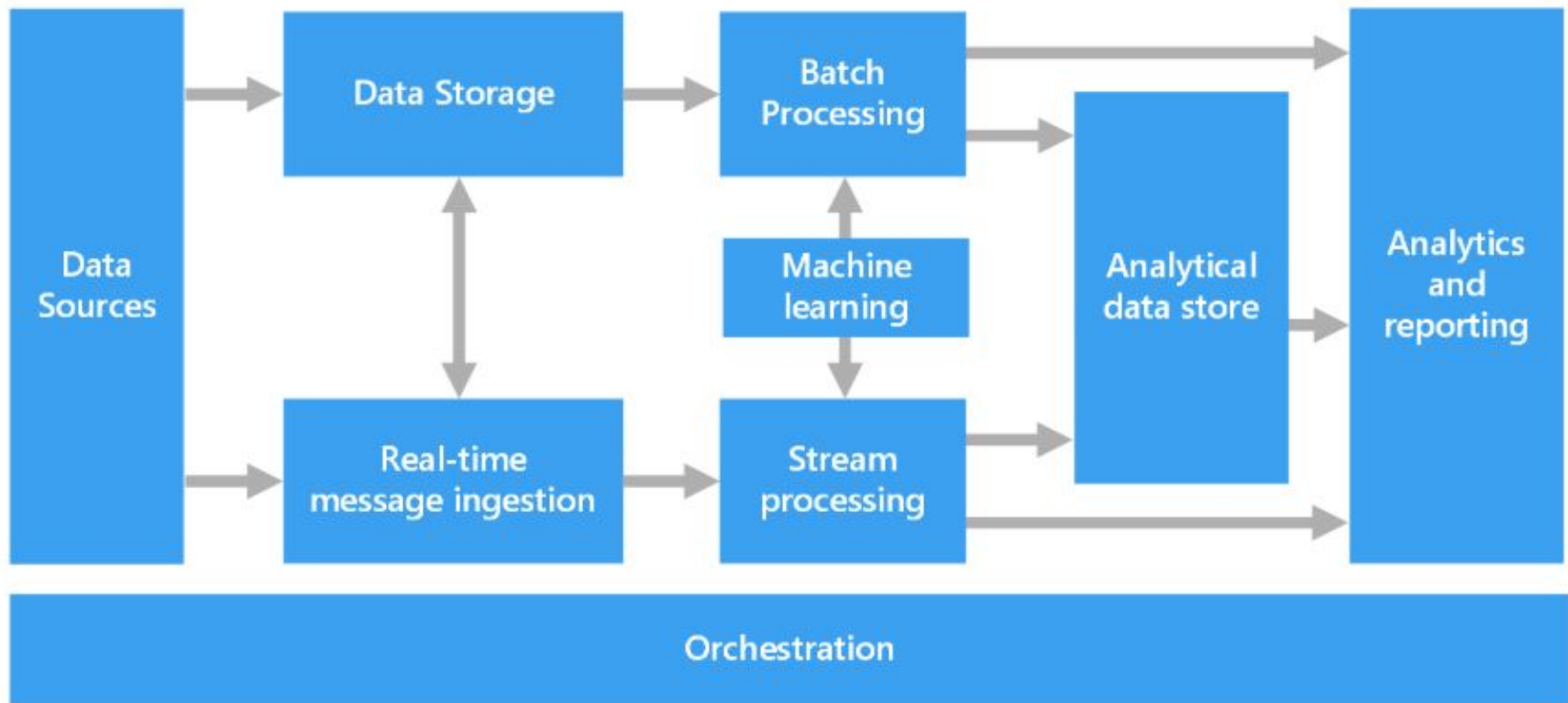
- No cenário da produção de dados, houve um grande salto na quantidade de dados que são produzidos nos últimos anos, bem como, a redução dos custos de armazenamento e processamento destes dados.
- Neste contexto, é possível perceber que os dados podem ser coletados de forma constante em pequenos *chunks* ou de forma mais espaçada com *chunks* muito grandes.
- Estes dados podem ser utilizados para análise de negócio, melhoria do processo, perfis dos usuários, algoritmos de aprendizado de máquina e IA, etc.

Big Data Architectures

- Soluções de Big Data geralmente envolvem um destes tipos de carga de processamento:
 - Processamento de dados em lote (batch processing).
 - Processamento de big data em real-time com dados dinâmicos que chegam a todo momento.
 - Exploração de dados de forma interativa.
 - Análises preditivas e aprendizado de máquina.
- Considere utilizar arquiteturas de Big Data quando:
 - Armazenar e processar grandes quantidades de dados quando comparado a um banco de dados tradicional.
 - Transformar grandes quantidades de dados não estruturados para análise e criação de relatórios.
 - Captar, processar e analisar diferentes fontes de dados em tempo real com baixa latência.

Big Data Architectures: Arquitetura Genérica

Analytical Data Storage é o componente de uma arquitetura de dados responsável por **armazenar dados já processados, estruturados e otimizados para consultas analíticas** — isto é, para gerar relatórios, dashboards, previsões e insights de negócio.



Big Data Architectures

- *Data sources*
 - Dados produzidos por aplicações e armazenados de forma local, geralmente bancos de dados relacionais.
 - Arquivos estáticos produzidos por aplicações, como arquivos de log.
 - Fontes de dados em real-time, por exemplo, dispositivos IoT.
- *Data storage*
 - Os dados usados em processamentos em lote são geralmente armazenados em sistemas distribuídos de grande capacidade, como os que compõem um data lake. O data lake é o repositório central de dados brutos, utilizado em arquiteturas de Big Data para processamentos em lote e em tempo real.
 - Exemplos: Azure Data Lake Store ou blob containers in Azure Storage; S3 na AWS e Redshift, etc.

Big Data Architectures

- Batch processing
 - Por trabalhar com grandes quantidades de dados, geralmente é preciso quebrar o processamento em diversos arquivos e distribuir o processamento em diferentes servidores.
 - Geralmente este tipo de processamento envolve: ler arquivos, processar e escrever a saída em novos arquivos. Exemplos de formatos de arquivos são: CSV, XML, JSON, Parquet, etc.
 - Geralmente aqui se utiliza estratégias de Map Reduce para quebrar os processamentos em diferentes servidores.
 - AWS possui ferramentas especializadas como AWS Batch Processing e a criação de servidores EMR (Elastic MapReduce).
 - coordena automaticamente um cluster inteiro de máquinas para processar grandes volumes de dados em paralelo.

Big Data Architectures

- Real-time message ingestion
 - Se a solução inclui fontes de dados de real-time, a arquitetura deve prover um meio para receber, processar e armazenar os dados em real-time.
 - Para este cenário, pode ser uma fila simples de mensagens ou o recebimento e armazenamento para processamento posterior.
 - Por exemplo, Azure Event Hubs, AWS SQS e Kafka ou RabbitMQ.
- Stream processing
 - Após a captura dos dados em real-time a solução deve processar os dados para filtragem, agregação de dados e preparação dos dados para análise.
 - O resultado deste processo pode ser escrito em outro local para ser analisado por outras ferramentas
 - Exemplo: Azure Stream Analytics e Apache Spark Streaming.

Big Data Architectures

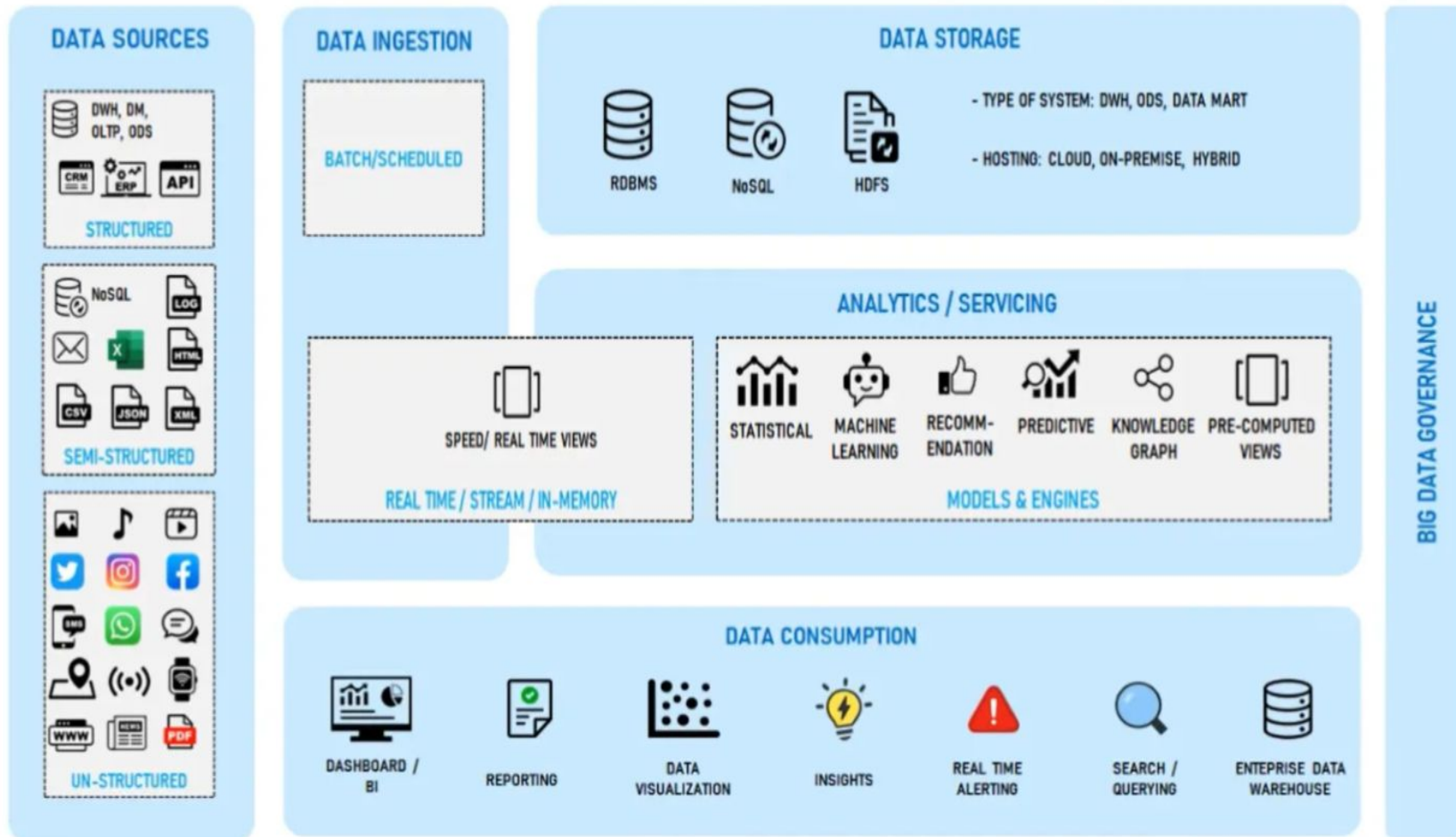
- Machine learning
 - Algoritmos de aprendizado de máquina podem ser usados a partir dos dados filtrados para criar modelos de predição ou classificação de dados. Estes modelos podem usar grandes bases de dados, onde os modelos resultantes podem ser utilizados para prever dados não presentes no conjunto de treinamento.
- Analytical data store
 - Muitas soluções de Big Data preparam os dados para análise pelos algoritmos e então servem estes dados já pré-processados. Algumas ferramentas permitem realizar consultas diretamente nos dados, inclusive em texto comum.
 - Algumas ferramentas permitem a criação de gráficos a partir de texto, por exemplo, “crie um gráfico para analisar a quantidade de vendas para o vendedor João no período dos últimos 5 anos”

Big Data Architectures

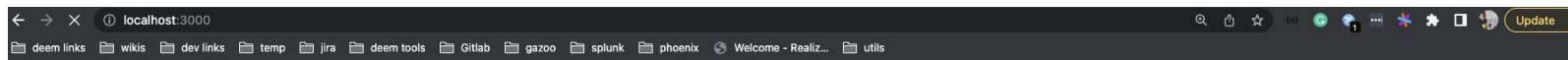
- Analysis and reporting
 - O objetivo da maioria das soluções para Big Data é prover ideias dos dados para análises e relatórios.
 - Para este cenário, é comum o uso de ferramentas de Business Intelligence (BI). Para esta fase pode-se usar Python ou R.
 - Para grandes bases de dados pode-se usar Spark.
- Orchestration
 - Por fim, a orquestração das operações. As operações geralmente consistem na repetição do processo de aquisição, processamento, análise e persistência de resultados.
 - Este processo pode ser automatizado com o Apache Airflow ou Azure Data Factory e Apache Oozie.

Big Data Architectures

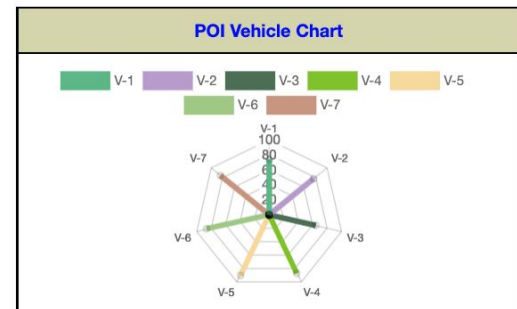
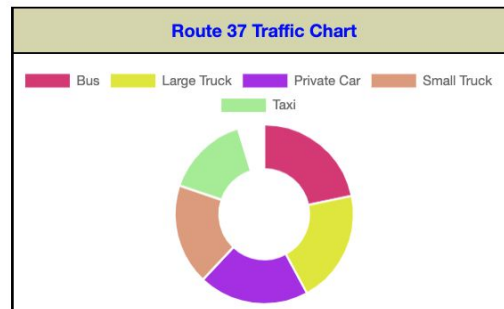
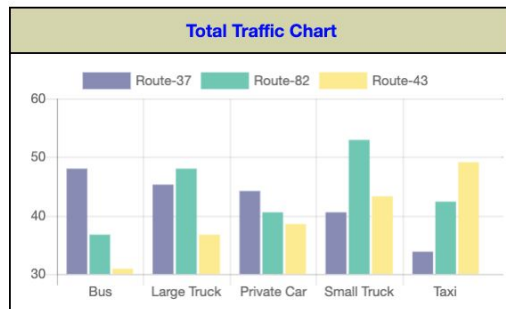
A very generic Big Data Architecture...



Big Data Architectures



IoT Traffic Data Monitoring Dashboard



Total Traffic Data

Route	Vehicle	Count	Time
Route-37	Bus	49	2022-04-19 15:16:47
Route-37	Large Truck	46	2022-04-19 15:18:12
Route-37	Private Car	45	2022-04-19 15:18:13

Last 30 Seconds Window

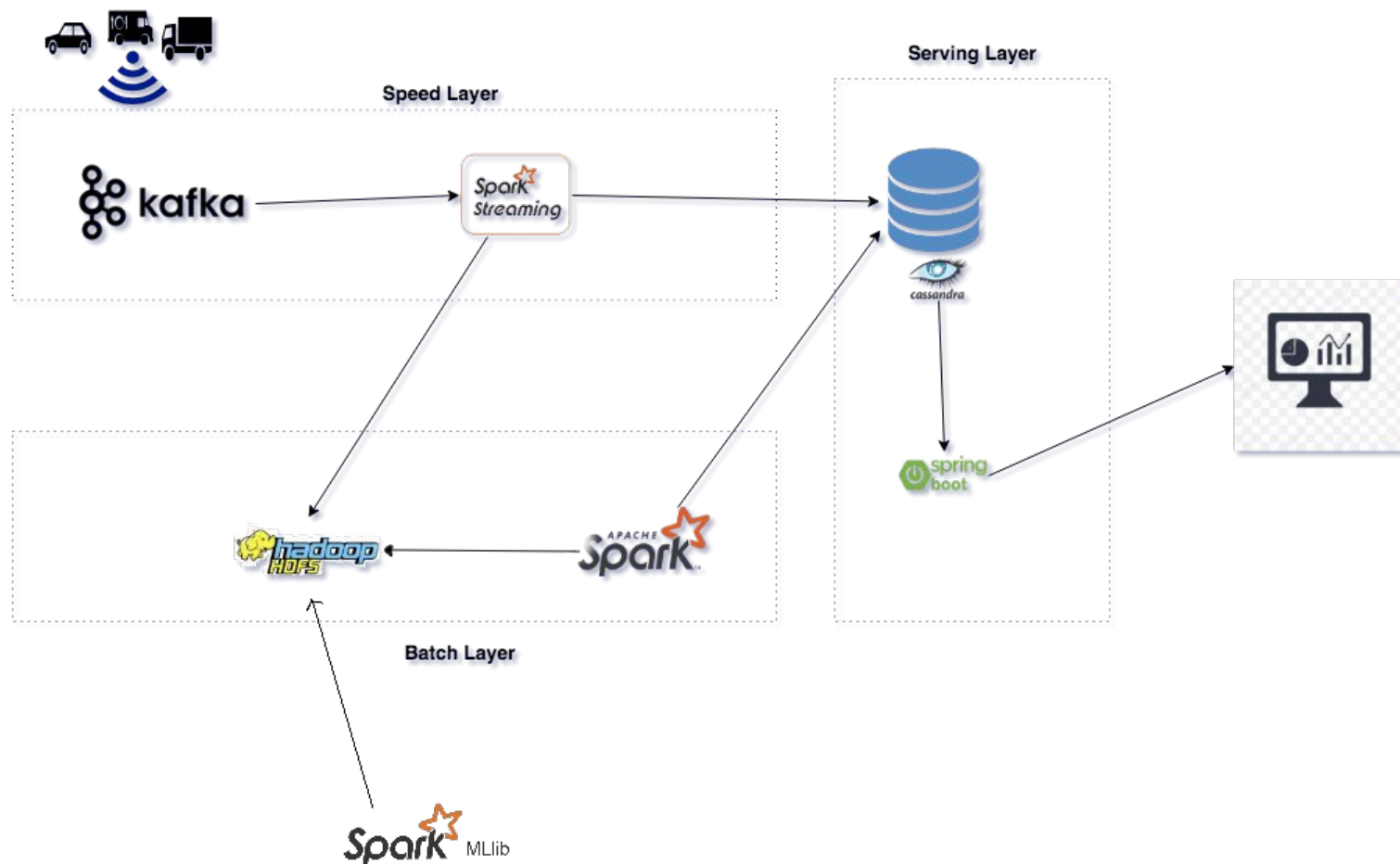
Route	Vehicle	Count	Time
Route-37	Bus	1	2022-04-19 15:17:04
Route-37	Large Truck	1	2022-04-19 15:18:13
Route-37	Private Car	1	2022-04-19 15:18:13

Trucks At POI

Vehicle Id	Vehicle	Distance	Time
9af6196a-e958-4b7a-b7b1-2dd9a92f704e	Taxi	77	2022-04-19 15:18:20
4356975a-	Large	81	2022-04-19 15:18:20

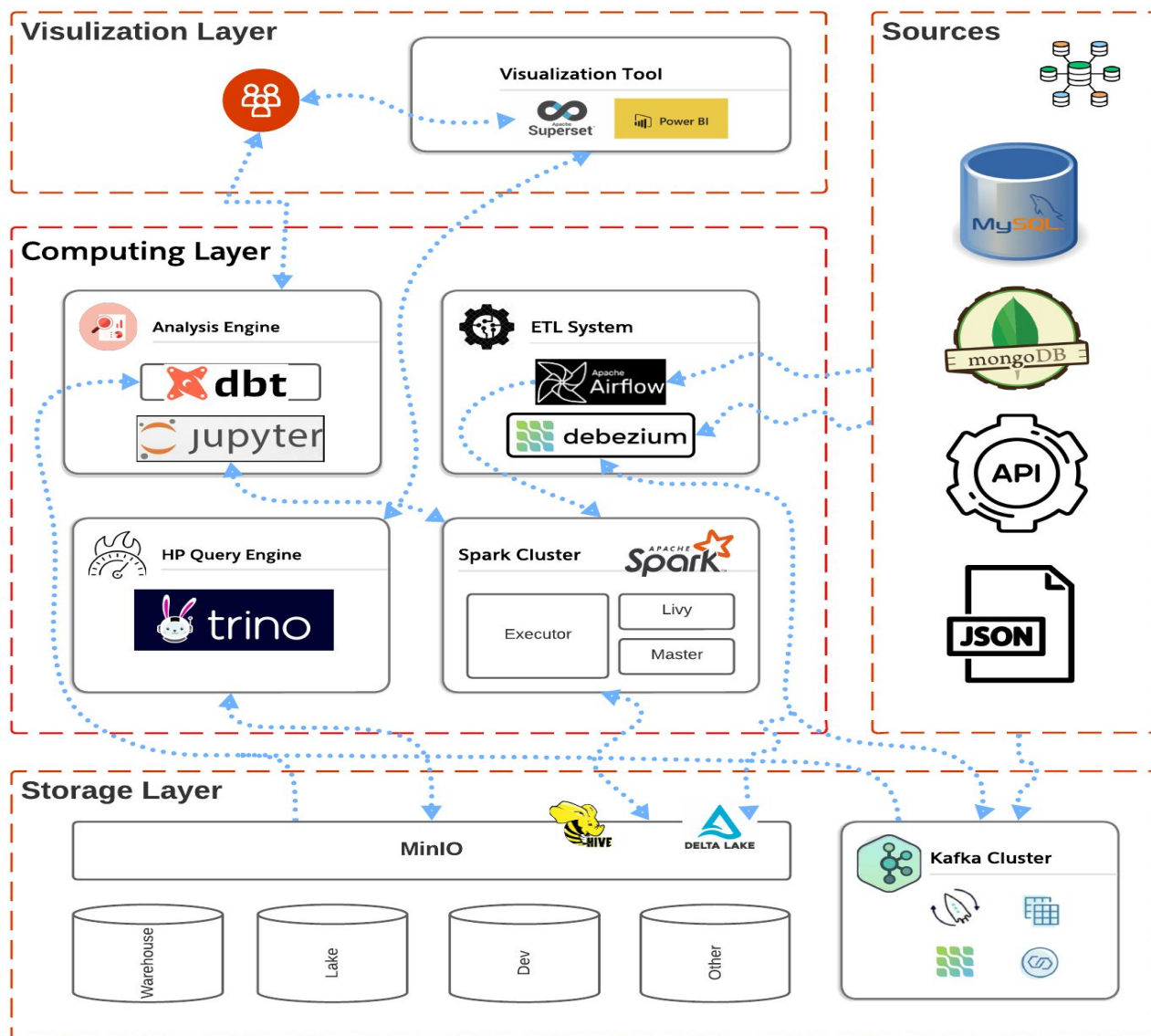
<https://github.com/apssouza22/big-data-pipeline-lambda-arch>

Big Data Architectures



<https://github.com/apssouza22/big-data-pipeline-lambda-arch>

Big Data Architectures



<https://github.com/dnguyenngoc/big-data>

Serverless Architecture

Serverless Architecture

- Neste estilo de arquitetura a criação e execução das aplicações ou módulos que compõem o sistema são abstraídos, não sendo necessário o gerenciamento dos servidores.
- Em contrapartida, é preciso configurar serviços na nuvem que serão gerenciados pela própria nuvem, por exemplo, o provisionamento, escalonamento e manutenção dos servidores é responsabilidade da nuvem hospedeira, não do time de desenvolvimento da aplicação.
- Como pode ser percebido, esse tipo de arquitetura é altamente escalável.

Serverless Architecture

- Backend como Serviço (BaaS)
 - Serviços prontos para funcionalidades específicas, como autenticação, banco de dados, notificações e armazenamento de arquivos.
 - Firebase, Supabase ou AWS Amplify.
- Funções como Serviço (FaaS)
 - Funções isoladas que são executadas sob demanda em resposta a eventos (como requisições HTTP ou alterações em um banco de dados).
 - AWS Lambda, Google Cloud Functions, ou Azure Functions.

Serverless Architecture

- Custo-Eficiência
 - Você paga apenas pelo tempo de execução do código e pelos recursos usados, sem custos fixos para manter servidores ociosos.
- Escalabilidade Automática
 - A infraestrutura ajusta automaticamente a quantidade de recursos para atender à demanda, desde picos altos até períodos de baixa atividade.
- Redução de Sobrecarga Operacional
 - Não é necessário configurar, monitorar ou atualizar servidores manualmente.
- Desenvolvimento mais Rápido
 - Você pode se concentrar na lógica do negócio em vez de se preocupar com infraestrutura.

Serverless Architecture

- Pontos não tão bons...
- Cold Starts
 - As funções serverless podem levar mais tempo para responder na primeira execução após ficarem inativas por um período. Precisam ser “levantadas” dado que estavam adormecidas para economizar recursos.
- Dependência de Provedor
 - Pode ser difícil migrar sua aplicação para outro provedor, pois muitos serviços utilizam APIs específicas. Por exemplo, AWS Lambda só vai funcionar na AWS :-). Embora você possa migrar o código não é algo tão fácil como subir um container Docker que independe do servidor.
- Monitoramento e Debugging
 - A natureza distribuída da arquitetura pode tornar o rastreamento de problemas mais complexo. Imagine ter que ler vários logs de diferentes serviços para saber onde quebrou e qual o caminho que foi feito pela requisição.

Serverless Architecture

- Exemplo sistema Serverless
- Visualizar Produtos
 - O frontend React faz uma requisição para uma função serverless (ex.: Lambda), que consulta o banco de dados e retorna a lista de produtos.
- Adicionar um Produto
 - O usuário insere os detalhes no frontend.
 - Uma requisição POST é feita para uma função que salva os dados no banco de dados.
- Editar Produto
 - Atualizações são feitas via uma requisição PUT para outra função serverless.
- Excluir Produto
 - Uma requisição DELETE aciona uma função para remover o produto do banco e sua imagem do armazenamento.

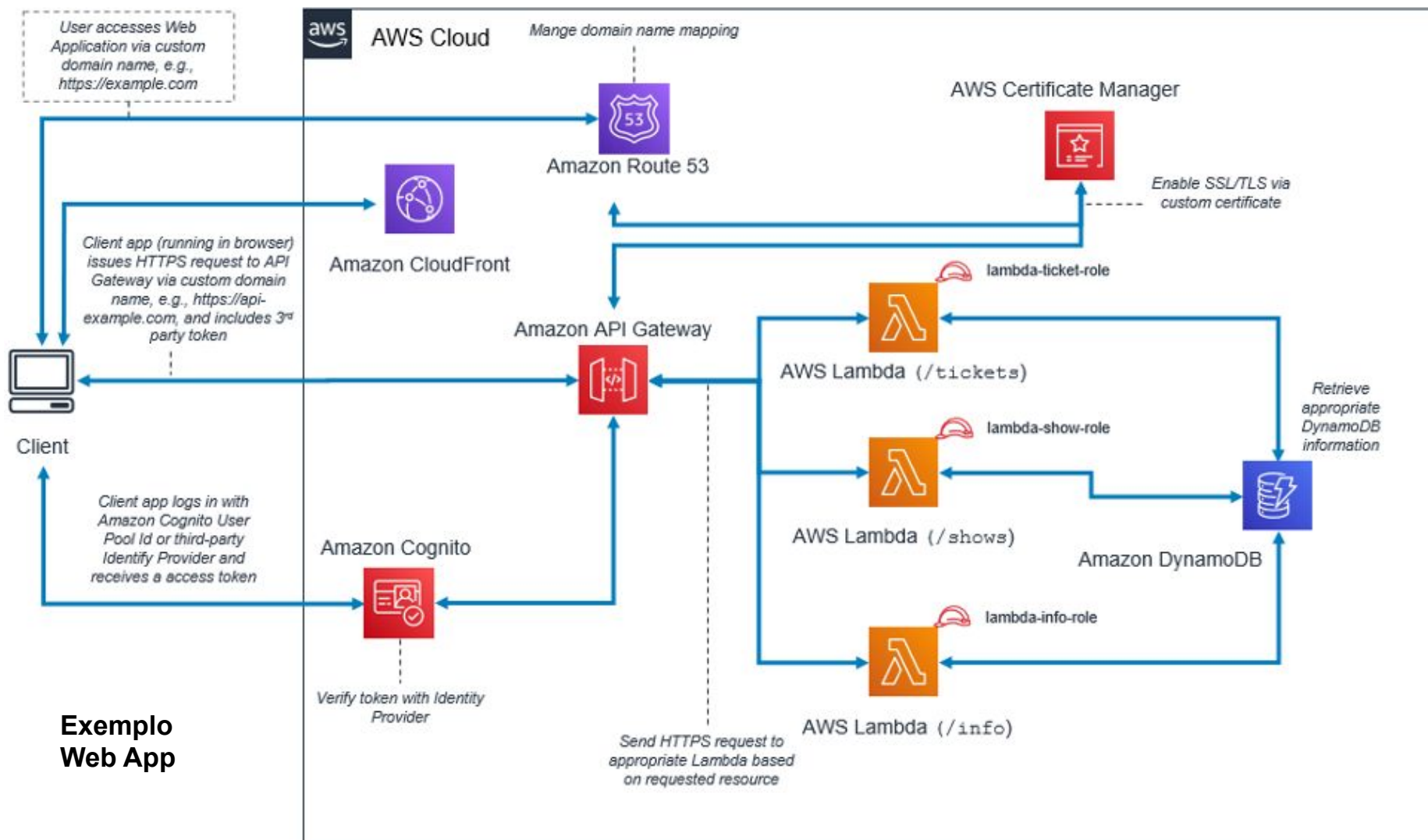
Serverless Architecture

- **Ferramentas e Tecnologias Sugeridas**
- Hospedagem do Frontend
 - Vercel, Netlify ou AWS Amplify.
- Backend
 - AWS Lambda, Google Cloud Functions, ou Firebase Cloud Functions.
- Banco de Dados
 - Firebase Firestore, DynamoDB, Aurora Serverless ou Atlas DB (mongo).
- Armazenamento de Imagens
 - AWS S3, Google Cloud ou Firebase Storage.
- Autenticação
 - Firebase Authentication ou AWS Cognito.

Serverless vs Microservices Architectures

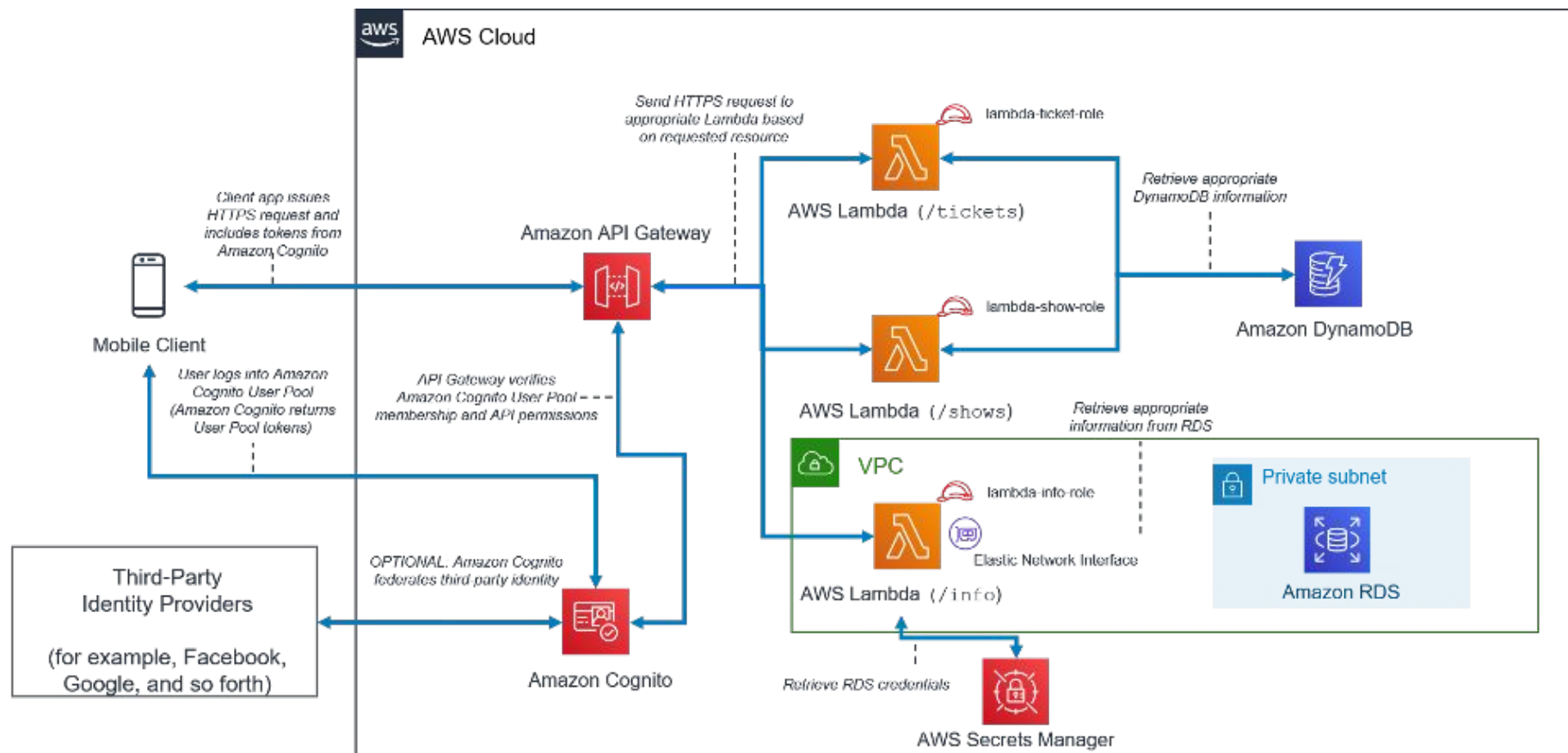
Aspecto	Microserviços	Serverless	Relação
Unidade lógica	Serviço	Função	Uma função pode ser vista como o "menor tipo" de microserviço
Infraestrutura	Containers (gerenciados por você)	Funções (gerenciadas pela nuvem)	Serverless abstrai infraestrutura
Escalabilidade	Manual ou via orquestrador (ex: Kubernetes)	Automática por evento	Serverless é mais granular
Estado	Pode ser stateful (banco acoplado)	Stateless (dados externos)	Funções precisam reconstituir contexto a cada execução
Comunicação	REST, gRPC, mensageria	Event-driven (filas, tópicos, gatilhos)	Serverless é fortemente orientado a eventos
Custos	Instâncias 24/7	Pay-per-use	Serverless é mais econômico para baixa demanda
Observabilidade	Log centralizado (ELK, Prometheus)	Logs distribuídos (CloudWatch, Stackdriver)	Difícil rastrear em Serverless
Gerenciamento	DevOps / SRE	Cloud provider	Serverless delega operação à nuvem

Serverless Architecture



<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

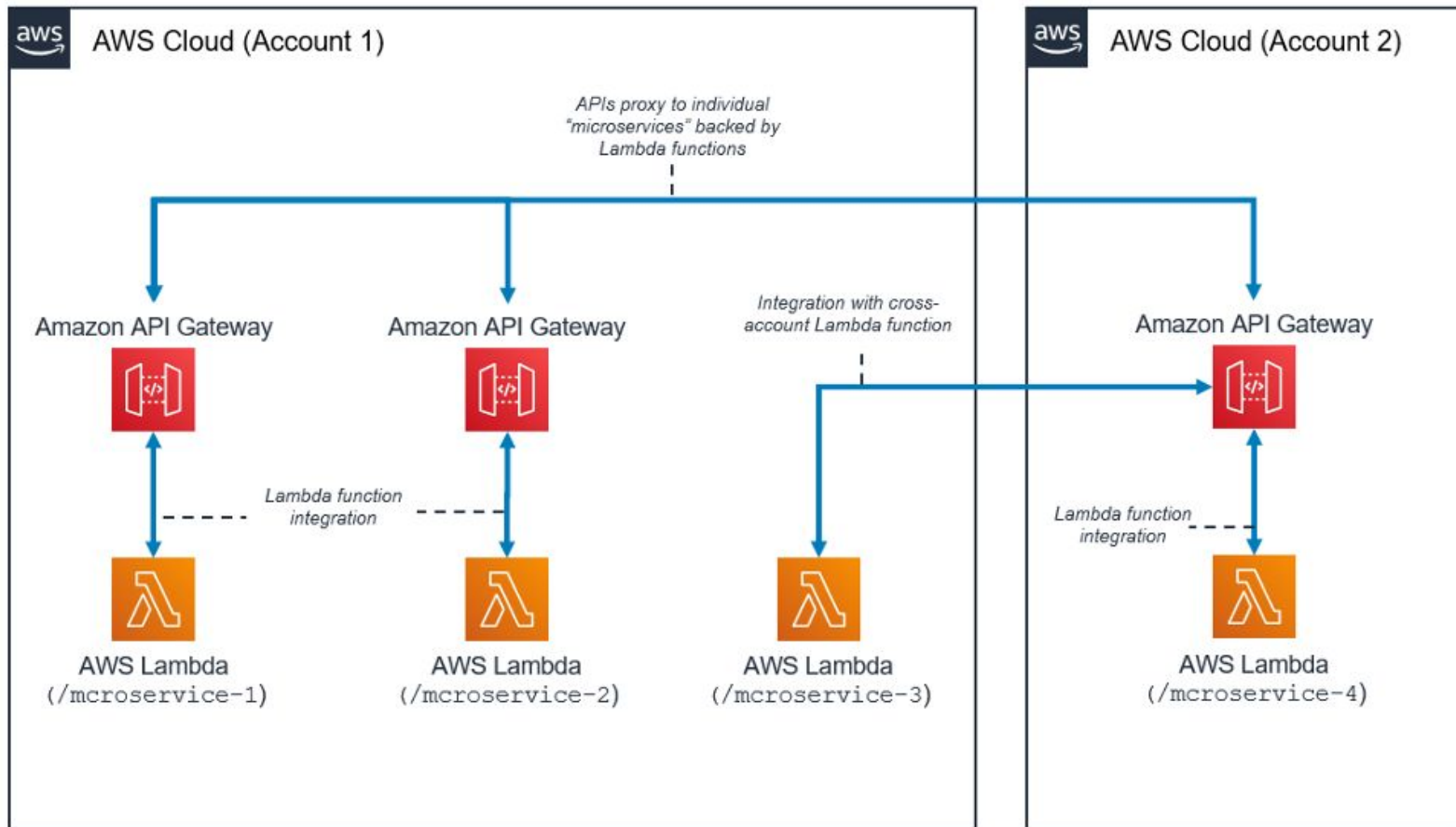
Serverless Architecture



**Mobile
backend**

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

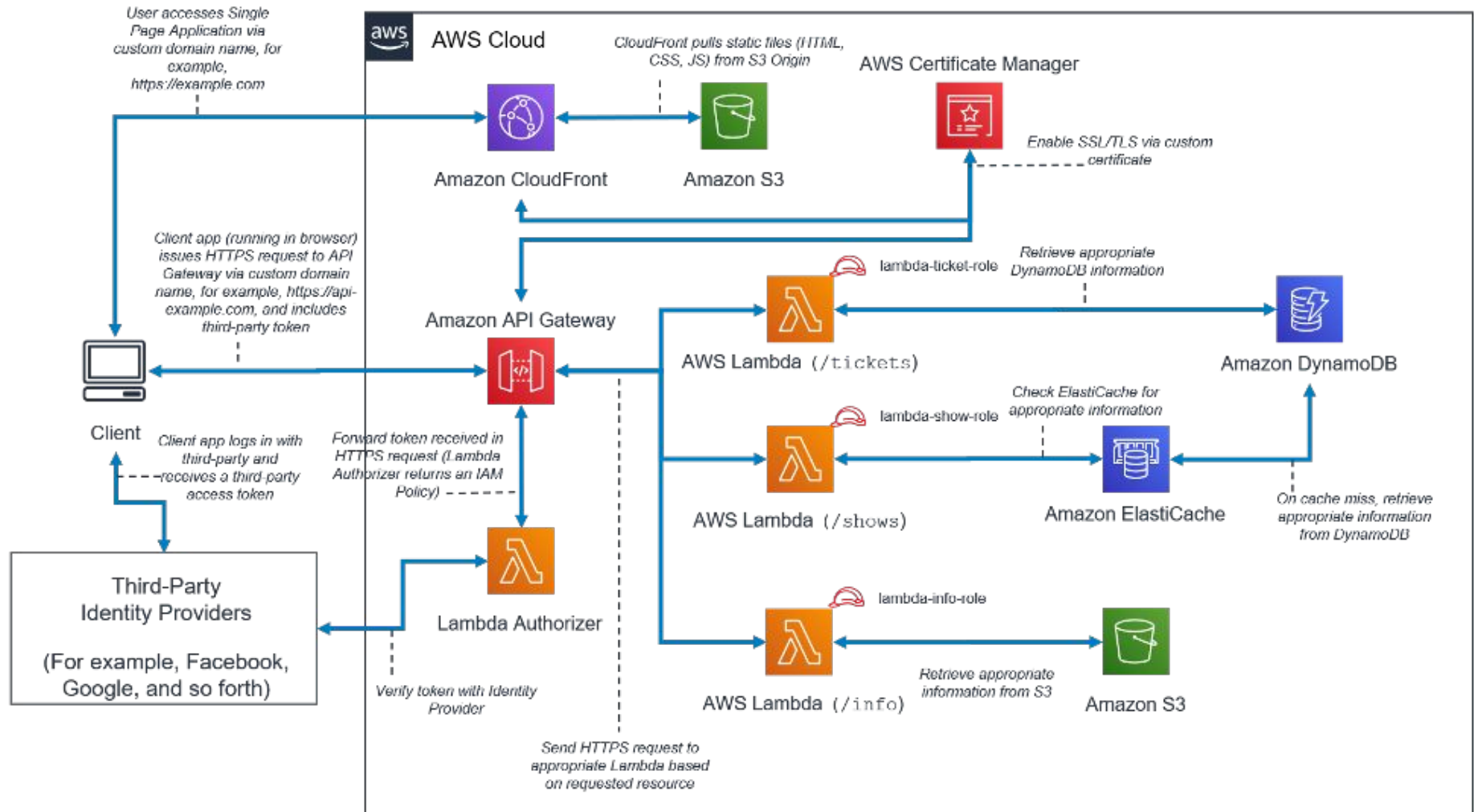
Serverless Architecture



Microserviços com lambda

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

Serverless Architecture



SPA

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

Serverless Architecture

- **E o que é o API Gateway da AWS?**
- O AWS API Gateway é um serviço gerenciado da Amazon Web Services que permite aos desenvolvedores criar, publicar, manter, monitorar e proteger APIs em escala.
- Ele é amplamente usado para expor endpoints de APIs REST, WebSocket e HTTP, possibilitando que diferentes aplicativos (web, móveis, etc.) se comuniquem de forma segura e eficiente com os serviços de backend.
- No contexto das arquiteturas serverless, pode-se criar um API Gateway para receber as requisições do usuário e mapear a requisição para uma determinada função lambda, chamando a função lambda de forma encapsulada, sem o conhecimento do usuário.

Serverless Architecture

- Exemplo prático
 - Máquinas de Venda Coca-cola
 - O cliente pode fazer o pedido no aplicativo do celular ou na própria máquina de refrigerantes.
 - Ao realizar o pedido, é iniciado um processo de autorização do crédito e liberação do refrigerante pela máquina.
 - Parece uma aplicação bem simples, certo?
 - E é, mas exige uma grande infraestrutura pela quantidade de requisições.
 - Quantas pessoas no mundo (ou em um país específico) estão comprando refrigerantes ao ir ou voltar do trabalho? Ao estar em passeio de lazer?



Serverless Architecture

- Ao migrar para uma arquitetura serverless na AWS, foi alcançado uma redução de 40% nos custos com infraestrutura.

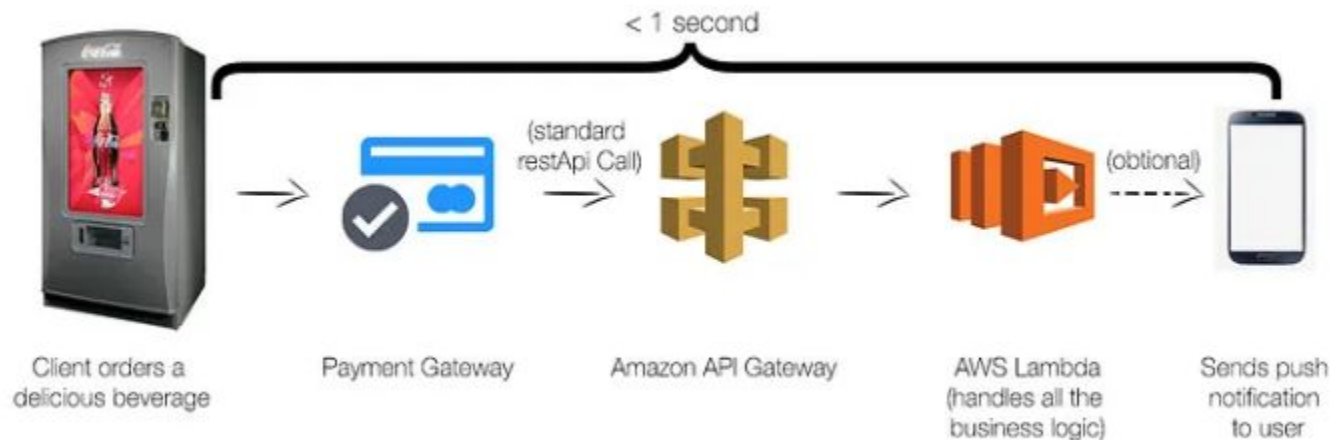
 Serverless in AWS				
	Managed NAT	replaces	Amazon EC2	costs \$0.045 per hour
	Lambda	replaces	EC2	costs \$0.00001667 per GB-s
	API Gateway	replaces	apigee, mashery	costs \$3.50 per MM calls + data
	Amazon SQS (queue)	replaces	messaging	costs \$0.60 per MM messages
	Amazon SES (email)	replaces	email providers	costs \$0.10 per 1000 messages
	Amazon SNS (notif.)	replaces	mobile provider	costs \$0.50 per MM SMS messages
	Amazon DynamoDB	replaces	big databases	costs \$180/mo for 36K writes per sec
	Amazon RDS (DB)	replaces	EC2 DBs	costs vary based on capacity

Serverless Architecture

- As máquinas automáticas de vendas de refrigerantes espalhadas pelo mundo possuem uma conexão com a central de processamentos da Coca-Cola.
- A lógica é bem simples, o cliente faz o pedido de uma bebida, o pagamento é processado pela Coca-Cola ou um parceiro que inicia uma requisição HTTP para uma API para liberar a bebida.

Serverless Architecture

- A Lambda da AWS é responsável por toda a lógica de negócio, similar ao back-end necessário para um processamento não-serverless.

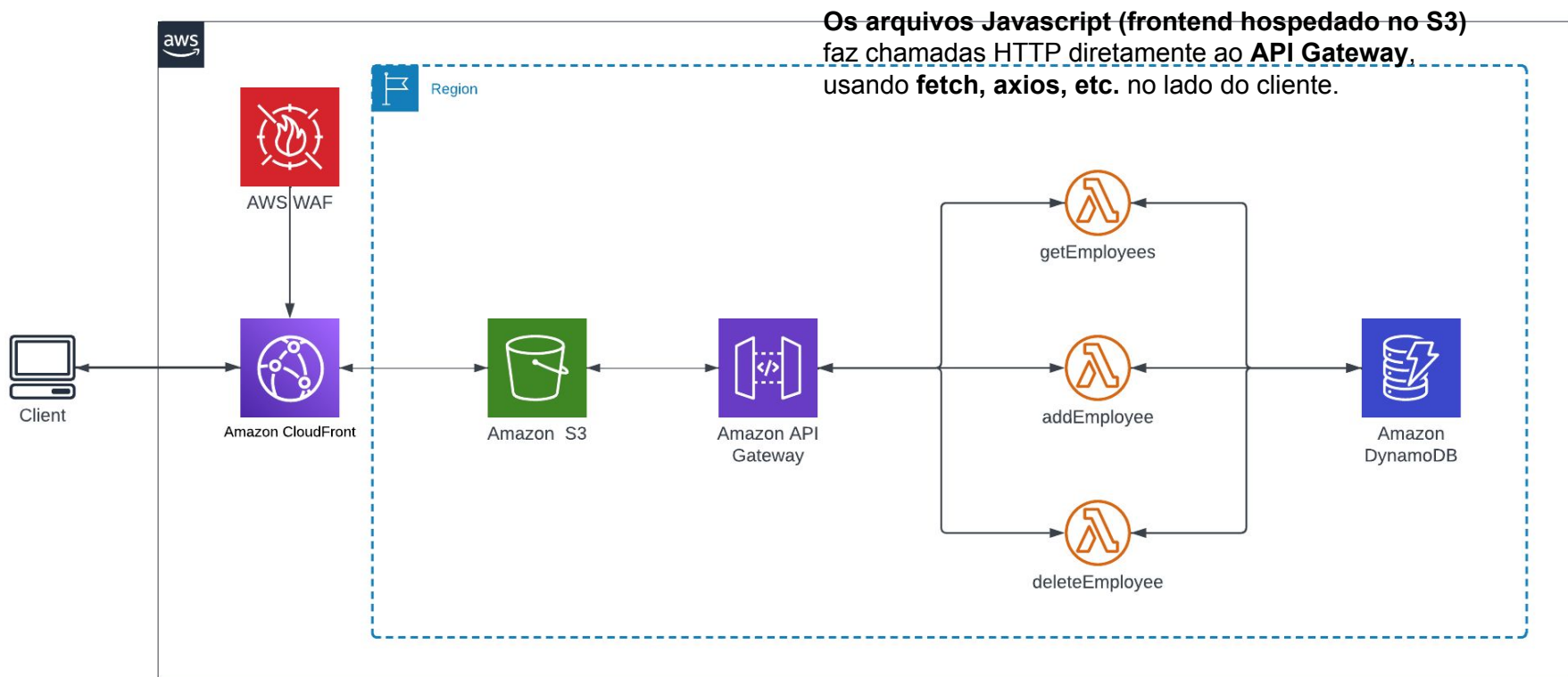


Se o usuário iniciou o pedido de um smartphone há um quinto passo, que é enviar uma notificação Push para o smartphone do usuário confirmando a transação.

Serverless Architecture

- O que mais poderia ser melhorado (ou inventado) com uma arquitetura de software *serverless*?
 - O que acham de um aplicativo para entrega das bebidas por drone?
 - Ou para melhorar os pedidos/distribuição das bebidas para os demais distribuidores (vendas no atacado)?
 - Como poderia ser essa arquitetura?

Serverless Architecture



Web (frontend): arquivos estáticos (HTML/CSS/JS, imagens) colocados em um bucket S3 e servidos globalmente via CloudFront (CDN). O `index.html` é o root do site.

API Gateway + Lambda (Functions): expõe endpoints REST (GET, POST, DELETE etc.). As Lambdas tratam as requisições e implementam a lógica de leitura/escrita.

DynamoDB: banco NoSQL usado para armazenar os dados da aplicação (itens usados pela API).

CloudFront + WAF: CloudFront acelera entrega do frontend; o WAF protege contra ataques/requests maliciosas (o README mostra como configurar regras gerenciadas).

<https://github.com/kemanedonfack/aws-serverless-web-application>

Serverless Architecture

<https://github.com/mikaelvesavuori/get-a-room-ddd-example/blob/main/readme/solution-diagram.png>

Dúvidas?



www.shutterstock.com · 744867163

Atividade

- Pesquisar e construir uma arquitetura serverless para algum sistema. Que tal pensar no PIX como um sistema serverless?
- Enviar no SIGA até próxima aula (20/01).
- A aula do dia 20/01 será reservada para a pesquisa e construção dessa arquitetura, não teremos aula presencial em sala. Usar o tempo da aula para fazer a atividade.
- Na aula seguinte, dia 21/01, sortearemos algumas equipes para apresentar a arquitetura no quadro, usando ferramenta ou desenhando no quadro.

